

JAVA 17 OCP · YDS STUDY SERIES

Unit 15 · JDBC · Bilingual Notes

Teknik İngilizceyi bağlam içinde öğren · kısa tekrarlarla kalıcılaştır

Bu ana kaynak, [OCP_Java_SE17_Chapter1den_Itibaren.pdf](#) içindeki ilgili chapter gövdesini ve Appendix resmî cevaplarını kaynak sırasını koruyan English → Türkçe paragraf çiftleriyle bir araya getirir. Kod ve terminal çıktıları çevrilmeden, bir kez ve kaynak konumunda gösterilir.

[Vocabulary](#) · [Grammar notes](#) · [Teknik hafıza notu](#)

Kaynak ve kapsam özeti

- Kaynak: [exam_lecture/OCP_Java_SE17_Chapter1den_Itibaren.pdf](#)
- Chapter: 15 · JDBC
- Chapter PDF sayfaları: 863–908
- Appendix cevap sayfaları: 959–961
- Beklenen kaynak sayfa sayısı: 46
- Beklenen resmî cevap: 21
- Eşleme biçimi: English paragraf → Türkçe çeviri → varsa kod

İçindekiler

- 1 [Introducing Relational Databases and SQL](#)
- 2 [Introducing the Interfaces of JDBC](#)
- 3 [Connecting to a Database](#)
- 4 [Working with a PreparedStatement](#)
- 5 [Getting Data from a ResultSet](#)
- 6 [Calling a CallableStatement](#)
- 7 [Controlling Data with Transactions](#)
- 8 [Closing Database Resources](#)
- 9 [Summary](#)
- 10 [Exam Essentials](#)
- 11 [Review Questions](#)
- 12 [Official Review Question Answers / Resmî Cevaplar](#)

Chapter 15 · JDBC · Eksiksiz çift dilli kaynak

Chapter 15 · JDBC

English: OCP exam objectives covered in this chapter: Accessing databases using JDBC.

Türkçe: Bu bölümde kapsanan OCP sınav hedefi: JDBC kullanarak veritabanlarına erişme.

English: Create connections, create and execute basic, prepared, and callable statements, process query results, and control transactions using the JDBC API.

Türkçe: *Connection* oluşturun; basic, prepared ve callable statement'lar oluşturup çalıştırın; query sonuçlarını işleyin ve JDBC API ile transaction'ları yönetin.

English: JDBC stands for Java Database Connectivity. This chapter introduces you to the basics of accessing databases from Java.

Türkçe: JDBC, Java Database Connectivity (Java Veritabanı Bağlantısı) anlamına gelir. Bu bölüm, Java'dan veritabanlarına erişmenin temellerini tanıtır.

English: We cover the key interfaces for how to connect, perform queries, process the results, and work with transactions.

Türkçe: Veritabanına bağlanma, query çalıştırma, sonuçları işleme ve transaction'larla çalışma süreçlerinde kullanılan temel interface'leri ele alacağız.

English: If you are new to JDBC, note that this chapter covers only the basics of JDBC and working with databases. What we cover is enough for the exam. To be ready to use JDBC on the job, we recommend that you read books on SQL along with Java and databases. For example, you might try *SQL for Dummies*, 9th edition, by Allen G. Taylor (Wiley, 2018) and *Practical Database Programming with Java* by Ying Bai (Wiley-IEEE Press, 2011).

Türkçe: JDBC'ye yeniyseniz bu bölümün yalnızca JDBC'nin ve veritabanlarıyla çalışmanın temellerini kapsadığını unutmayın. Anlatılanlar sınav için yeterlidir. JDBC'yi iş hayatında kullanmaya hazırlanmak için Java ve veritabanı kaynaklarının yanında SQL kitapları da okumanızı öneririz. Örneğin Allen G. Taylor'ın *SQL for Dummies*, 9. baskısını (Wiley, 2018) ve Ying Bai'nin *Practical Database Programming with Java* kitabını (Wiley-IEEE Press, 2011) inceleyebilirsiniz.

For Experienced Developers

English: If you are an experienced developer and know JDBC well, you can skip the “Introducing Relational Databases and SQL” section. Read the rest of this chapter carefully, though. We found that the exam covers some topics that developers don't use in practice, in particular, these:

Türkçe: Deneyimli bir geliştiriciyseniz ve JDBC'yi iyi biliyorsanız “Introducing Relational Databases and SQL” bölümünü atlayabilirsiniz. Yine de chapter'ın geri kalanını dikkatlice okuyun. Sınavın, geliştiricilerin pratikte kullanmadıkları bazı konuları, özellikle de şunları kapsadığını gördük:

English: You probably set up the URL once for a project for a specific database. Often, developers just copy and paste it from somewhere else. For the exam, you have to understand this rather than rely on looking it up.

Türkçe: Belirli bir veritabanı için JDBC URL'yi bir projede muhtemelen yalnızca bir kez ayarlarsınız. Geliştiriciler URL'yi çoğu zaman başka bir yerden kopyalayıp yapıştırır. Sınavda ise URL'yi bir kaynaktan aramak yerine yapısını anlamanız gerekir.

English: You are likely using a DataSource. For the exam, you have to remember or relearn how DriverManager works.

Türkçe: Büyük olasılıkla bir DataSource kullanıyorsunuzdur. Sınav için DriverManager'in nasıl çalıştığını hatırlamanız veya yeniden öğrenmeniz gerekir.

Introducing Relational Databases and SQL

English: Data is information. A piece of data is one fact, such as your first name. A database is an organized collection of data. In the real world, a file cabinet is a type of database. It has file folders, each of which contains pieces of paper. The file folders are organized in some way, often alphabetically. Each piece of paper is like a piece of data. Similarly, the folders on your computer are like a database. The folders provide organization, and each file is a piece of data.

Türkçe: Veri bilgidir. Bir veri parçası, ilk adınız gibi bir gerçektir. Veri tabanı, organize bir veri koleksiyonudur. Gerçek dünyada, bir dosya dolabı bir tür veritabanıdır. Her biri kağıt parçaları içeren dosya klasörleri vardır. Dosya klasörleri bir şekilde, genellikle alfabetik olarak düzenlenir. Her kağıt parçası bir veri parçası gibidir. Benzer şekilde, bilgisayarınızdaki klasörler bir veritabanı gibidir. Klasörler organizasyon sağlar ve her dosya bir veri parçasıdır.

English: A relational database is a database that is organized into tables, which consist of rows and columns. You can think of a table as a spreadsheet. There are two main ways to access a relational database from Java:

Türkçe: Relational database (ilişkisel veritabanı), row ve column'lardan oluşan table'lar halinde düzenlenmiş bir veritabanıdır. Bir table'ı elektronik tablo gibi düşünebilirsiniz. Java'dan ilişkisel bir veritabanına erişmenin iki ana yolu vardır:

English: Java Database Connectivity (JDBC): Accesses data as rows and columns. JDBC is the API covered in this chapter.

Türkçe: Java Database Connectivity (JDBC): Verilere row ve column olarak erişir. Bu chapter'da ele alınan API JDBC'dir.

English: Java Persistence API (JPA): Accesses data through Java objects using a concept called object-relational mapping (ORM). The idea is that you don't have to write as much code, and you get your data in Java objects. JPA is not on the exam, and therefore it is not covered in this chapter.

Türkçe: Java Persistence API (JPA): Object-relational mapping (ORM; nesne-ilişkisel eşleme) kavramını kullanarak verilere Java object'leri üzerinden erişir. Böylece daha az code yazılır ve veriler Java object'leri halinde alınır. JPA sınav kapsamında değildir; bu nedenle bu chapter'da ele alınmaz.

English: A relational database is accessed through Structured Query Language (SQL). SQL is a programming language used to interact with database records. JDBC works by sending a SQL command to the database and then processing the response.

Türkçe: İlişkisel bir veritabanına Structured Query Language (SQL; Yapılandırılmış Sorgu Dili) üzerinden erişilir. SQL, veritabanı kayıtlarıyla etkileşim kurmak için kullanılan bir dildir. JDBC, veritabanına bir SQL komutu gönderip gelen yanıtı işleyerek çalışır.

English: In addition to relational databases, there is another type of database called a NoSQL database. These databases store their data in a format other than tables, such as key/value, document stores, and graph-based databases. NoSQL is out of scope for the exam as well.

Türkçe: İlişkisel veritabanlarının yanında NoSQL database adı verilen başka bir veritabanı türü de vardır. Bu veritabanları verilerini key/value, document store ve graph-based database gibi table dışındaki biçimlerde saklar. NoSQL de sınav kapsamı dışındadır.

English: In the following sections, we introduce a small relational database that we will be using for the examples in this chapter and present the SQL to access it. We also cover some vocabulary that you need to know.

Türkçe: Sonraki bölümlerde, chapter örneklerinde kullanacağımız küçük bir relational database'i ve ona erişmek için gereken SQL'i tanıtıyoruz. Ayrıca bilmeniz gereken bazı terimleri ele alıyoruz.

Running the Examples in the Chapter

English: In most chapters of this book, you need to write code and try lots of examples. This chapter is different. It's still nice to try the examples, but you can probably get the JDBC questions correct on the exam from just reading this chapter and mastering the review questions.

Türkçe: Kitabın çoğu chapter'ında code yazmanız ve pek çok örnek denemeniz gerekir. Bu chapter farklıdır. Örnekleri denemek yararlı olsa da yalnızca bu chapter'ı okuyup review question'larda ustalaşarak sınavdaki JDBC sorularını doğru yanıtlatabilirsiniz.

English: While the exam is database agnostic, we had to use a database for the examples, and we chose the HyperSQL database. It is a small, in-memory database. In fact, you need only one JAR file to run it. For real projects, we like MySQL and PostgreSQL.

Türkçe: Sınav belirli bir veritabanına bağlı değildir; ancak örnekler için bir veritabanı seçmemiz gerektiğinden küçük ve in-memory çalışan HyperSQL'i kullandık. Çalıştırmak için yalnızca bir JAR dosyası gerekir. Gerçek projelerde ise MySQL ve PostgreSQL'i tercih ediyoruz.

English: Instructions to download and set up the database for the examples in the chapter are in:

Türkçe: Bölümdeki örnekler için veritabanını indirmek ve kurmak için talimatlar şunlardır:

www.selikoff.net/ocp17

English: For now, you don't need to understand any of the code on the website. It is just to get you set up. In a nutshell, it connects to the database and creates two tables. By the end of this chapter, you should understand how to create a Connection and PreparedStatement in this manner.

Türkçe: Şimdilik web sitesindeki code'u anlamamız gerekmez; amaç yalnızca ortamı kurmanızı sağlamaktır. Özetle bu code veritabanına bağlanır ve iki table oluşturur. Chapter'ın sonunda bu biçimde Connection ve PreparedStatement oluşturmayı anlayacaksınız.

English: There are plenty of tutorials for installing and getting started with any of these. It's beyond the scope of the book and the exam to set up a database, but feel free to ask questions in the database/JDBC section of CodeRanch. You might even get an answer from the authors.

Türkçe: Bu veritabanlarından herhangi birini kurup kullanmaya başlamak için çok sayıda tutorial vardır. Database kurulumu kitap ve sınav kapsamı dışındadır; ancak CodeRanch'ın database/JDBC bölümünde soru sorabilirsiniz. Hatta yazarlardan biri yanıtlayabilir.

Identifying the Structure of a Relational Database

English: Our sample database has two tables. One has a row for each species that is in our zoo. The other has a row for each animal. These two relate to each other because an animal belongs to a species. These relationships are why this type of database is called a relational database.

Türkçe: Örnek veritabanımızda iki table vardır. Birinde hayvanat bahçemizdeki her species için, diğerinde ise her animal için birer row bulunur. Animal bir species'e ait olduğu için bu iki table birbiriyle ilişkilidir. Bu ilişkiler, bu veritabanı türüne relational database denmesinin nedenidir.

English: FIGURE 15.1 shows the structure of our database.

Türkçe: FIGURE 15.1 veritabanımızın yapısını gösterir.

English: FIGURE 15.1 · Tables in our relational database.

Türkçe: FIGURE 15.1 · Relational database'imizdeki table'lar.

Database: Zoo · Table: exhibits

id · integer · primary key	name · varchar(255)	num_acres · numeric
1	African Elephant	7.5
2	Zebra	1.2

Database: Zoo · Table: names

id · integer · primary key	species_id · integer	name · varchar(255)
1	1	Elsa
2	2	Zelda
3	1	Ester

id · integer · primary key	species_id · integer	name · varchar(255)
4	1	Eddie
5	2	Zoe

names.species_id → exhibits.id
foreign reference primary key

English: As you can see in FIGURE 15.1, we have two tables. One is named exhibits, and the other is named names. Each table has a primary key, which gives us a unique way to reference each row. After all, two animals might have the same name, but they can't have the same ID. You don't need to know about keys for the exam. We mention them to give you a bit of context. In our example, it so happens that the primary key is only one column. In some situations, it is a combination of columns called a compound key. For example, a student identifier and year might be a compound key.

Türkçe: FIGURE 15.1'de görüldüğü gibi `exhibits` ve `names` adlı iki table vardır. Her table'ın bir primary key'i bulunur; bu anahtar her row'a benzersiz biçimde başvurmamızı sağlar. Sonuçta iki animal aynı name'e sahip olabilir, ancak aynı ID'ye sahip olamaz. Sınav için key'leri bilmeniz gerekmez; yalnızca bağlam sağlamak için söz ediyoruz. Örneğimizde primary key tek bir column'dur. Bazı durumlarda compound key (bileşik anahtar) adı verilen bir column kombinasyonu kullanılır. Örneğin student identifier ile year birlikte compound key oluşturabilir.

English: There are two rows and three columns in the exhibits table and five rows and three columns in the names table. You do need to know about rows and columns for the exam.

Türkçe: `exhibits` table'ında iki row ve üç column; `names` table'ında beş row ve üç column vardır. Sınav için row ve column kavramlarını bilmeniz gerekir.

Writing Basic SQL Statements

English: The most important thing that you need to know about SQL for the exam is that there are four types of statements for working with the data in tables. They are referred to as CRUD (Create, Read, Update, Delete). The SQL keywords don't match the acronym, so pay attention to the SQL keyword for each in TABLE 15.1.

Türkçe: Sınav için SQL hakkında bilmeniz gereken en önemli şey, tablolardaki verilerle çalışmak için dört tür ifadenin olmasıdır. CRUD olarak adlandırılırlar (Create, Read, Update, Delete). SQL anahtar kelimeler kısaltmayla eşleşmez, bu nedenle TABLE 15.1'deki her biri için SQL anahtar kelimesine dikkat edin.

English: TABLE 15.1 · CRUD operations.

Türkçe: TABLE 15.1 CRUD işlemleri.

Operation	SQL keyword	Description
Create	INSERT	Adds a new row to a table
Read	SELECT	Retrieves data from a table
Update	UPDATE	Changes zero or more rows in a table
Delete	DELETE	Removes zero or more rows from a table

English: That's it. You are not expected to determine whether SQL statements are correct. You are not expected to spot syntax errors in SQL statements. You are not expected to write SQL statements. Notice a theme?

Türkçe: İşte bu. SQL ifadelerinin doğru olup olmadığını belirlemeniz beklenmez. SQL ifadelerinde sözdizimi hatalarını fark etmeniz beklenmez. SQL ifadeleri yazmanız beklenmez. Bir tema fark ettin mi?

English: Unlike Java, SQL keywords are case insensitive. This means select, SELECT, and Select are all equivalent. Like Java primitive types, SQL has a number of data types. Most are self-explanatory, like INTEGER. There's also DECIMAL, which functions a lot like a double in Java. The strangest one is VARCHAR, standing for "variable character," which is like a String in Java. The variable part means that the database should use only as much space as it needs to store the value.

Türkçe: Java'dan farklı olarak SQL keyword'leri case-insensitive'dir. Bu nedenle select, SELECT ve Select eşdeğerdir. Java'nın primitive type'ları gibi SQL'in de çeşitli data type'ları vardır. INTEGER gibi çoğu kendini açıklar. DECIMAL, Java'daki double gibi çalışır. "Variable character" anlamındaki VARCHAR ise Java'daki Stringe benzer; variable kısmı, veritabanının değeri saklamak için yalnızca gereken kadar alan kullanacağını belirtir.

English: While you don't have to know how to write them, we present the basic four SQL statements in TABLE 15.2 since they appear in many questions.

Türkçe: Bunları nasıl yazacağınızı bilmek zorunda olmamakla birlikte, birçok soruda göründükleri için TABLE 15.2'de temel dört SQL ifadesini sunuyoruz.

English: TABLE 15.2 · Basic SQL statements.

Türkçe: TABLE 15.2 Temel SQL ifadeleri.

SQL	Explanation
INSERT INTO exhibits VALUES (3, 'Asian Elephant', 7.5);	Adds a new row with the provided values. Defaults to the order in which columns were defined in the table.
SELECT * FROM exhibits WHERE ID = 3;	Reads data with an optional WHERE clause. SELECT can return all columns with *, named columns, or a function such as COUNT(*).

TABLE 15.2 · continued

SQL	Explanation
<pre>UPDATE exhibits SET num_acres = num_acres + .5 WHERE name = 'Asian Elephant';</pre>	Sets a column's value with an optional WHERE clause to limit the rows updated.
<pre>DELETE FROM exhibits WHERE name = 'Asian Elephant';</pre>	Deletes rows with an optional WHERE clause to limit the rows deleted.

Introducing the Interfaces of JDBC

English: For the exam, you need to know five key interfaces of JDBC. The interfaces are declared in the JDK. They are just like all of the other interfaces and classes that you've seen in this book. For example, in Chapter 9, "Collections and Generics," you worked with the interface `List` and the concrete class `ArrayList`.

Türkçe: Sınav için JDBC'nin beş temel interface'ini bilmeniz gerekir. Bu interface'ler JDK'da declare edilmiştir ve kitaptaki diğer interface ve class'lar gibidir. Örneğin Chapter 9'da, "Collections and Generics" konusunda `List` interface'i ve concrete `ArrayList` class'ı ile çalıştınız.

English: With JDBC, the concrete classes come from the JDBC driver. Each database has a different JAR file with these classes. For example, PostgreSQL's JAR is called something like `postgresql-9.4-1201.jdbc4.jar`. MySQL's JAR is called something like `mysql-connector-java-5.1.36.jar`. The exact name depends on the vendor and version of the driver JAR.

Türkçe: JDBC'de concrete class'lar JDBC driver'ından gelir. Her veritabanı bu class'ları içeren farklı bir JAR dosyası sağlar. Örneğin PostgreSQL JAR'ı `postgresql-9.4-1201.jdbc4.jar`, MySQL JAR'ı ise `mysql-connector-java-5.1.36.jar` benzeri bir ad taşır. Tam ad, vendor'a ve driver JAR'ının sürümüne bağlıdır.

English: This driver JAR contains an implementation of these key interfaces along with a number of other interfaces. The key is that the provided implementations know how to communicate with a database. There are also different types of drivers; luckily, you don't need to know about this for the exam.

Türkçe: Bu driver JAR'ı, temel interface'lerin ve başka birçok interface'in implementation'larını içerir. Önemli nokta, sağlanan implementation'ların veritabanıyla nasıl iletişim kuracağını bilmesidir. Farklı driver türleri de vardır; ancak sınav için bunları bilmeniz gerekmez.

English: FIGURE 15.2 shows the five key interfaces that you need to know. It also shows that the implementation is provided by an imaginary Foo driver JAR. They cleverly stick the name Foo in all classes.

Türkçe: FIGURE 15.2, bilmeniz gereken beş temel interface'i gösterir. Ayrıca uygulamanın hayali bir Foo sürücüsü JAR tarafından sağlandığını gösterir. Foo adını tüm sınıflara akıllıca yapıştırırlar.

English: You've probably noticed that we didn't tell you what the implementing classes are called in any real database. The main point is that you shouldn't know. With JDBC, you use only the interfaces in your code and never the implementation classes directly. In fact, they might not even be public classes.

Türkçe: Muhtemelen, uygulama sınıflarının herhangi bir gerçek veritabanında ne olarak adlandırıldığını söylemediğimizi fark ettiniz. Önemli olan, bilmemeniz gerektiridir. JDBC ile, yalnızca kodunuzdaki interface'leri kullanırsınız ve asla uygulama sınıflarını doğrudan kullanmazsınız. Hatta public sınıfı bile olmayabilirler.

English: What do these five interfaces do? On a very high level, we have the following:

Türkçe: Bu beş interface ne yapar? Çok yüksek bir seviyede, aşağıdakiler var:

English: Driver: Establishes a connection to the database Connection: Sends commands to a database PreparedStatement: Executes a SQL query CallableStatement: Executes commands stored in the database ResultSet: Reads the results of a query

Türkçe: **Driver:** Veritabanı bağlantısını kurar. **Connection:** Veritabanına komut gönderir. **PreparedStatement:** Bir SQL query'sini çalıştırır. **CallableStatement:** Veritabanında saklanan komutları çalıştırır. **ResultSet:** Bir query'nin sonuçlarını okur.

English: FIGURE 15.2 · Key JDBC interfaces. The interfaces are in the JDK; their concrete implementations are supplied by the database driver.

Türkçe: FIGURE 15.2 Anahtar JDBC interface'leri. Interface'ler JDK içerisinde; somut uygulamaları database driver tarafından sağlanır.

Interfaces in the JDK	Implementation in the imaginary Foo driver
Driver	→ FooDriver
Connection	→ FooConnection
PreparedStatement	→ FooPreparedStatement
CallableStatement	→ FooCallableStatement
ResultSet	→ FooResultSet

English: All database interfaces are in the package `java.sql`, so we often omit the imports throughout this chapter.

Türkçe: Tüm veritabanı interface'leri `java.sql` package'ındadır; bu nedenle chapter boyunca import bildirimlerini çoğu zaman göstermeyiz.

English: In this next example, we show you what JDBC code looks like, end to end. If you are new to JDBC, just notice that three of the five interfaces are in the code. If you are experienced, remember that the exam uses the `DriverManager` class instead of the `DataSource` interface.

Türkçe: Sonraki örnek, JDBC code'unun baştan sona nasıl görüldüğünü gösterir. JDBC'ye yeniyseniz beş interface'ten üçünün code'da yer aldığına dikkat edin. Deneyimliyseniz sınavın `DataSource` yerine `DriverManager` class'ını kullandığını unutmayın.

```
public class MyFirstDatabaseConnection {
    public static void main(String[] args) throws SQLException {
        String url = "jdbc:hsqldb:file:zoo";
        try (Connection conn = DriverManager.getConnection(url);
            PreparedStatement ps = conn.prepareStatement(
                "SELECT name FROM exhibits");
            ResultSet rs = ps.executeQuery()) {
            while (rs.next())
                System.out.println(rs.getString(1));
        } } }
```

English: If the URL were using our imaginary Foo driver, DriverManager would return an instance of FooConnection. Calling prepareStatement() would then return an instance of FooPreparedStatement, and calling executeQuery() would return an instance of FooResultSet. Since the URL uses hsqldb instead, it returns the implementations that HyperSQL has provided for these interfaces. You don't need to know their names. In the rest of the chapter, we explain how to use all five of the interfaces and go into more detail about what they do. By the end of the chapter, you'll be writing code like this yourself.

Türkçe: URL, hayalî Foo driver'ını kullansaydı DriverManager bir FooConnection instance'ı döndürürdü. Ardından prepareStatement() bir FooPreparedStatement, executeQuery() ise bir FooResultSet instance'ı döndürürdü. URL bunun yerine hsqldb kullandığından, HyperSQL'in bu interface'ler için sağladığı implementation'lar döner. Bunların adlarını bilmeniz gerekmez. Chapter'ın devamında beş interface'in tamamını ve görevlerini ayrıntılı biçimde öğreneceksiniz.

English: Compiling with Modules Almost all the packages on the exam are in the java.base module. As you may recall from Chapter 12, "Modules," this module is included automatically when you run your application as a module.

Türkçe: Modüllerle Derleme Sınavdaki hemen hemen tüm paketler java.base modülündedir. Bölüm 12'den hatırlayabileceğiniz gibi, "Modüller", uygulamanızı bir modül olarak çalıştırdığınızda bu modül otomatik olarak dahil edilir.

English: In contrast, the JDBC classes are all in the module java.sql. They are also in the package java.sql. The names are the same, so they should be easy to remember. When working with SQL, you need the java.sql module and import java.sql.*.

Türkçe: Buna karşılık tüm JDBC class'ları java.sql module'ündedir ve aynı adlı java.sql package'ında bulunur. Adlar aynı olduğu için hatırlamak kolaydır. SQL ile çalışırken java.sql module'üne ve import java.sql.*; bildirimine ihtiyaç duyarsınız.

English: We recommend separating your studies for JDBC and modules. You can use the classpath when working with JDBC and reserve your practice with the module path for when you are studying modules.

Türkçe: JDBC ve module çalışmalarını ayrı yürütmenizi öneririz. JDBC çalışırken classpath'i, module çalışırken ise module path'i kullanabilirsiniz.

English: That said, if you do want to use JDBC code with modules, remember to update your module-info file to include the following:

Türkçe: Yine de JDBC code'unu module'lerle kullanmak istiyorsanız `module-info.java` dosyanıza aşağıdaki bildirimi eklemeyi unutmayın:

```
requires java.sql;
```

Connecting to a Database

English: The first step in doing anything with a database is connecting to it. First we show you how to build the JDBC URL. Then we show you how to use it to get a Connection to the database.

Türkçe: Bir veritabanı ile herhangi bir şey yapmanın ilk adımı ona bağlanmaktır. İlk olarak JDBC URL nasıl oluşturulacağını göstereceğiz. Ardından, veritabanına bir `Connection` almak için nasıl kullanılacağını gösteririz.

Building a JDBC URL

English: To access a website, you need to know its URL. To access your email, you need to know your username and password. JDBC is no different. To access a database, you need to know this information about it.

Türkçe: Bir web sitesine erişmek için URL adresini bilmeniz gerekir. E-postanıza erişmek için kullanıcı adınızı ve şifrenizi bilmeniz gerekir. JDBC farklı değil. Bir veritabanına erişmek için, bununla ilgili bu bilgileri bilmeniz gerekir.

English: Unlike web URLs, JDBC URLs have a variety of formats. They have three parts in common, as shown in FIGURE 15.3. The first piece is always the same. It is the protocol `jdbc`. The second part is the subprotocol, which is the name of the database, such as `hsqldb`, `mysql`, or `postgres`. The third part is the subname, which is a database-specific format. Colons (:) separate the three parts.

Türkçe: Web URL'lerinden farklı olarak JDBC URL'leri çeşitli biçimlerde olabilir; fakat FIGURE 15.3'te gösterilen üç ortak parçaya sahiptir. İlk parça her zaman `jdbc` protocol'üdür. İkinci parça `hsqldb`, `mysql` veya `postgres` gibi veritabanının adını belirten subprotocol'dür. Üçüncü parça, veritabanına özgü biçimde yazılan subname'dir. Bu üç parçayı colon (:) ayırır.

English: The subname typically contains information about the database such as its location and/or name. The syntax varies. You need to know about the three main parts. You don't need to memorize the subname formats. Phew! You've already seen one such URL:

Türkçe: Subname genellikle veritabanının konumu ve/veya adı gibi bilgileri içerir; syntax'ı değişebilir. Üç ana parçayı bilmeniz gerekir, fakat subname biçimlerini ezberlemeniz gerekmez. Daha önce böyle bir URL gördünüz:

```
jdbc:hsqldb:file:zoo
```

English: FIGURE 15.3 · The JDBC URL format.

Türkçe: FIGURE 15.3 JDBC URL biçimi.

```
j d b c : h s q l d b : // l o c a l h o s t : 5 4 3 2 / z o o
|   |   |   |
p r o t o c o l   s u b p r o t o c o l   s u b n a m e
                  p r o d u c t / v e n d o r   d a t a b a s e - s p e c i f i c   c o n n e c t i o n   d e t a i l s
                  c o l o n   s e p a r a t o r s
```

English: Notice the three parts. It starts with jdbc, and then comes the subprotocol `hsqldb`. It ends with the subname, which tells us we are using the file system. The location is then the database name.

Türkçe: Üç kısma dikkat edin. `jdbc` ile başlar ve ardından `hsqldb` subprotocol'ü gelir. File system kullandığımızı söyleyen alt isimle sona erer. Konum daha sonra veritabanı adıdır.

Figure 15.3 editor note: Figure'daki `//localhost:5432/zoo` subname'i network host/port/database ayrıntısıdır; paragraftaki “using the file system” ifadesi bu illustrated URL ile uyuşmaz. File-based örnek, hemen önce verilen `jdbc:hsqldb:file:zoo` URL'sidir.

English: Other examples of subnames are shown here:

Türkçe: Alt isimlerin diğer örnekleri burada gösterilmiştir:

```
j d b c : p o s t g r e s q l : // l o c a l h o s t / z o o
j d b c : o r a c l e : t h i n : @ 1 2 3 . 1 2 3 . 1 2 3 . 1 2 3 : 1 5 2 1 : z o o
j d b c : m y s q l : // l o c a l h o s t : 3 3 0 6
j d b c : m y s q l : // l o c a l h o s t : 3 3 0 6 / z o o ? p r o f i l e S Q L = t r u e
```

English: You can see that each of these JDBC URLs begins with jdbc, followed by a colon, followed by the vendor/product name. After that, the URLs vary. Notice how all of them include the location of the database: localhost, 123.123.123.123:1521, and localhost:3306. Also, notice that the port is optional when using the default location.

Türkçe: Bu JDBC URL'lerinin her birinin `jdbc` ile başlayıp colon'dan sonra vendor/product adını içerdiğini görebilirsiniz. Bundan sonra URL'ler değişir. Hepsinin veritabanının konumunu nasıl içerdiğine dikkat edin: localhost, 123.123.123.123:1521 ve `localhost:3306`. Ayrıca default konum kullanıldığında port bilgisinin optional olduğuna dikkat edin.

Getting a Database Connection

English: There are two main ways to get a Connection: DriverManager and DataSource. DriverManager is the one covered on the exam. Do not use a DriverManager in code someone is paying you to write. A DataSource has more features than DriverManager. For example, it can pool connections or store the database connection information outside the application.

Türkçe: Connection almak için iki ana yol vardır: DriverManager ve DataSource. Sınavda DriverManager ele alınır. Production code'da ise DataSource tercih edilir; çünkü DriverManager'dan daha fazla özellik sunar. Örneğin connection pooling yapabilir veya database connection bilgilerini application dışında saklayabilir.

English: The DriverManager class is in the JDK, as it is an API that comes with Java. It uses the factory pattern, which means that you call a static method to get a Connection rather than calling a constructor. As you saw in Chapter 11, "Exceptions and Localization," the factory pattern means that you can get any implementation of the interface when calling the method. The good news is that the method has an easy-to-remember name: getConnection().

Türkçe: DriverManager, Java ile gelen bir API olduğu için JDK'da yer alır. Factory pattern kullanır: constructor çağırmak yerine static bir method çağırarak Connection alırsınız. Chapter 11'de gördüğünüz gibi factory pattern, method çağırısından interface'in herhangi bir implementation'ının dönebilmesini sağlar. Method'un hatırlaması kolay bir adı vardır: getConnection().

English: To get a Connection from the HyperSQL database, you write the following:

Türkçe: HyperSQL veritabanından bir Connection almak için aşağıdaki code yazılır:

```
import java.sql.*;
public class TestConnect {
    public static void main(String[] args) throws SQLException {
        try (Connection conn =
```

```
            DriverManager.getConnection("jdbc:hsqldb:file:zoo")) {
            System.out.println(conn);
        }
    }
}
```

English: As in Chapter 11, we use a try-with-resources statement to ensure that database resources are closed. We cover closing database resources in more detail later in the chapter. We also throw the checked SQLException, which means something went wrong. For example, you might have forgotten to set the location of the database driver in your classpath.

Türkçe: Chapter 11'de olduğu gibi database resource'larının kapanmasını sağlamak için try-with-resources statement kullanıyoruz. Database resource'larını kapatmayı chapter'ın ilerleyen kısmında ayrıntılı ele alacağız. Ayrıca bir şeylerin ters gittiğini bildiren checked SQLException fırlatıyoruz. Örneğin database driver'ının konumunu classpath'te belirtmeyi unutmuş olabilirsiniz.

English: Assuming the program runs successfully, it prints something like this:

Türkçe: Programın başarılı bir şekilde çalıştığını varsayarsak, şöyle bir şey yazdırır:

```
org.hsqldb.jdbc.JDBCConnection@3dfc5fb8
```

English: The details of the output aren't important. Just notice that the class is not `Connection`. It is a vendor implementation of `Connection`.

Türkçe: Çıktının ayrıntıları önemli değildir. Class'ın `Connection` olmadığına dikkat edin; bu, `Connection` interface'inin vendor tarafından sağlanan bir implementation'dır.

English: There is also a signature that takes a username and password.

Türkçe: Username ve password alan bir overload signature da vardır.

```
import java.sql.*;
public class TestExternal {
    public static void main(String[] args) throws SQLException {
        try (Connection conn = DriverManager.getConnection(
            "jdbc:postgresql://localhost:5432/ocp-book",
            "username",
            "Password20182")) {
            System.out.println(conn);
        }
    }
}
```

English: Notice the three parameters that are passed to `getConnection()`. The first is the JDBC URL that you learned about in the previous section. The second is the username for accessing the database, and the third is the password for accessing the database. It should go without saying that our password is not `Password20182`. Also, don't put your password in real code. It's a horrible practice. Always load it from some kind of configuration, ideally one that keeps the stored value encrypted.

Türkçe: `getConnection()` a geçirilen üç parameter'a dikkat edin. Birincisi önceki bölümde öğrendiğiniz JDBC URL, ikincisi veritabanına erişmek için username, üçüncüsü ise password'dür. Gerçek parolanızın `"Password20182"` olmadığını söylemeye gerek yok. Ayrıca, şifrenizi gerçek koda koymayın. Korkunç bir uygulamadır. Her zaman bir tür yapılandırmadan yükleyin, ideal olarak depolanan değeri şifreli tutan bir yapılandırma.

English: If you were to run this with the Postgres driver JAR, it would print something like this:

Türkçe: Bunu Postgres driver JAR'ı ile çalıştırırsanız aşağıdakine benzer bir çıktı alınır:

```
org.postgresql.jdbc4.Jdbc4Connection@eed1f14
```

English: Again, notice that it is a driver-specific implementation class. You can tell from the package name. Since the package is `org.postgresql.jdbc4`, it is part of the PostgreSQL driver.

Türkçe: Bunun yine driver'a özgü bir implementation class olduğuna dikkat edin. Bunu package adından anlayabilirsiniz: `org.postgresql.jdbc4` package'ında olduğundan PostgreSQL driver'ının bir parçasıdır.

English: Unless the exam specifies a command line, you can assume that the correct JDBC driver JAR is in the classpath. The exam creators explicitly ask about the driver JAR if they want you to think about it.

Türkçe: Soru bir command line belirtmedikçe doğru JDBC driver JAR'ının `classpath`'te olduğunu varsayabilirsiniz. Sınav hazırlayıcıları driver JAR'ını değerlendirmenizi istiyorsa bunu soruda açıkça belirtir.

English: The nice thing about the factory pattern is that it takes care of the logic of creating a class for you. You don't need to know the name of the class that implements `Connection`, and you don't need to know how it is created. You are probably a bit curious, though.

Türkçe: Fabrika deseninin güzel yanı, sizin için bir sınıf yaratma mantığıyla ilgilenmesidir. `Connection` uygulayan sınıfın adını bilmenize gerek yok ve nasıl oluşturulduğunu bilmenize gerek yok. Ama muhtemelen biraz meraklısındır.

English: `DriverManager` looks through any drivers it can find to see whether they can handle the JDBC URL. If so, it creates a `Connection` using that Driver. If not, it gives up and throws a `SQLException`.

Türkçe: `DriverManager`, bulabildiği driver'ları tarayarak JDBC URL'yi hangisinin işleyebileceğini belirler. Uygun bir `Driver` bulursa onunla `Connection` oluşturur; bulamazsa `SQLException` fırlatır.

English: You might see `Class.forName()` in code. It was required with older drivers (that were designed for older versions of JDBC) before getting a `Connection`.

Türkçe: Code içinde `Class.forName()` görebilirsiniz. Eski JDBC sürümleri için tasarlanmış driver'larda `Connection` alınmadan önce bu çağrı gerekiyordu.

Working with a PreparedStatement

English: In Java, you have a choice of working with a `Statement`, `PreparedStatement`, or `CallableStatement`. The latter two are subinterfaces of `Statement`, as shown in FIGURE 15.4.

Türkçe: Java'da `Statement`, `PreparedStatement` veya `CallableStatement` ile çalışabilirsiniz. FIGURE 15.4'te gösterildiği gibi son ikisi `Statement`'in subinterface'idir.

English: FIGURE 15.4 · Types of statements.

Türkçe: FIGURE 15.4 · *Statement* türleri.

```
Statement
├── PreparedStatement
└── CallableStatement
```

Editör notu · Java 17: Kaynak şekil iki alt interface'i yan yana gösterir; burada doğrudan inheritance ilişkisi açıklanmıştır. *CallableStatement*, *PreparedStatement*ı extend eder; *PreparedStatement* da *Statement*ı extend eder. Böylece *CallableStatement*, *Statement*ın dolaylı bir alt interface'idir. [Java 17 CallableStatement bildirimi](#).

English: Later in the chapter, you learn about using *CallableStatement* for queries that are inside the database. In this section, we look at *PreparedStatement*.

Türkçe: Chapter'ın ilerleyen kısmında veritabanı içinde saklanan query'ler için *CallableStatement* kullanmayı öğreneceksiniz. Bu bölümde *PreparedStatement*ı inceliyoruz.

English: What about *Statement*, you ask? It is an interface that both *PreparedStatement* and *CallableStatement* extend. A *Statement* and a *PreparedStatement* are similar to each other, except that a *PreparedStatement* takes parameters, while a *Statement* does not. A *Statement* just executes whatever SQL query you give it.

Türkçe: Peki *Statement* nedir? *PreparedStatement* ve *CallableStatement*ın extend ettiği bir interface'tir. *Statement* ile *PreparedStatement* birbirine benzer; ancak *PreparedStatement* parametre alırken *Statement* almaz. *Statement*, kendisine verilen SQL query'sini doğrudan çalıştırır.

English: While it is possible to run SQL directly with *Statement*, you shouldn't. *PreparedStatement* is far superior for the following reasons:

Türkçe: SQL'i doğrudan *Statement* ile çalıştırmak mümkün olsa da bunu yapmamalısınız. *PreparedStatement* aşağıdaki nedenlerle çok daha üstündür:

English: Performance: In most programs, you run similar queries multiple times. When you use *PreparedStatement*, the database software often devises a plan to run the query well and remembers it.

Türkçe: Performans: Çoğu programda, benzer sorguları birden fazla kez çalıştırırsınız. *PreparedStatement* kullandığınızda, veritabanı yazılımı genellikle sorguyu iyi çalıştırmak için bir plan tasarlar ve hatırlar.

English: Security: You are protected against an attack called SQL injection when using a *PreparedStatement* correctly.

Türkçe: Güvenlik: *PreparedStatement* doğru kullanıldığında SQL injection saldırılarına karşı korunursunuz.

English: Readability: It's nice not to have to deal with string concatenation in building a query string with lots of parameters.

Türkçe: Okunabilirlik: Çok sayıda parametrelili bir sorgu dizisi oluşturmada dize birleştirme ile uğraşmak zorunda kalmamak güzel bir şey.

English: Future use: Even if your query is being run only once or doesn't have any parameters, you should still use a `PreparedStatement`. That way, future editors of the code won't add a variable and have to remember to change to `PreparedStatement` then.

Türkçe: Gelecekte kullanım: Query yalnızca bir kez çalıştırılırsa veya hiç parameter içermese bile `PreparedStatement` kullanmalısınız. Böylece code'u gelecekte düzenleyen biri bir variable eklediğinde yapıyı sonradan `PreparedStatement`a dönüştürmeyi hatırlamak zorunda kalmaz.

Little Bobby Tables

English: SQL injection is often caused by a lack of properly sanitized user input. The author of the popular xkcd.com, web-comic once asked the question, what would happen if someone's name contained a SQL statement?

Türkçe: SQL injection genellikle uygun biçimde sanitize edilmemiş user input eksikliğinden kaynaklanır. Popüler xkcd.com, web-comic'in yazarı bir kez şu soruyu sordu: Birinin adı bir SQL ifadesi içeriyorsa ne olur?

English: "Exploits of a Mom" reproduced with permission from xkcd.com/327/ Oops! Guess the school should have used a `PreparedStatement` and bound each student's name to a variable. If it had, the entire String would have been properly escaped and stored in the database.

Türkçe: "Exploits of a Mom", xkcd.com/327 izniyle çoğaltılmıştır. Görünüşe göre okul `PreparedStatement` kullanıp her öğrencinin adını bir variable'a bind etmeliydi. Bunu yapsaydı `String`in tamamı doğru biçimde escape edilerek veritabanında saklanırdı.

English: Using the `Statement` interface directly is not in scope for the JDBC exam, so we do not cover it in this book. In the following sections, we cover obtaining a `PreparedStatement`, executing one, working with parameters, and running multiple updates.

Türkçe: `Statement` interface'ini doğrudan kullanmak JDBC sınavı kapsamında değildir; bu nedenle kitapta ele alınmaz. Sonraki bölümlerde `PreparedStatement` elde etmeyi ve çalıştırmayı, parameter'larla çalışmayı ve birden çok update yürütmeyi öğreneceğiz.

Obtaining a PreparedStatement

English: To run SQL, you need to tell a `PreparedStatement` about it. Getting a `PreparedStatement` from a `Connection` is easy.

Türkçe: SQL çalıştırmak için SQL metnini bir `PreparedStatement`a vermeniz gerekir. `Connection`dan `PreparedStatement` almak kolaydır.

```
try (PreparedStatement ps = conn.prepareStatement(
    "SELECT * FROM exhibits")) {
    // work with ps
}
```

English: An instance of a `PreparedStatement` represents a SQL statement that you want to run using the `Connection`. It does not execute the query yet! We get to that shortly.

Türkçe: Bir `PreparedStatement` instance'ı, `Connection` üzerinden çalıştırmak istediğiniz SQL statement'ını temsil eder; query'yi henüz çalıştırmaz. Çalıştırma adımına birazdan geleceğiz.

English: Passing a SQL statement when creating the object is mandatory. You might see a trick on the exam.

Türkçe: `Object` oluşturulurken SQL statement'ını geçirmek zorunludur. Sınavda bu noktaya yönelik bir tuzakla karşılaşabilirsiniz.

```
try (var ps = conn.prepareStatement()) { // DOES NOT COMPILE
}
```

English: The previous example does not compile, because SQL is not supplied at the time a `PreparedStatement` is requested. We also used `var` in this example. We write JDBC code both using `var` and the actual class names to get you used to both approaches.

Türkçe: SQL, `PreparedStatement` istenirken sağlanmadığı için önceki örnek derlenmez. Bu örnekte `var` kullandık. İki yaklaşıma da alışmanız için JDBC code'unu hem `var` hem de gerçek class adlarıyla yazıyoruz.

English: There are overloaded signatures that allow you to specify a `ResultSet` type and concurrency mode. On the exam, you only need to know how to use the default options, which process the results in order.

Türkçe: `ResultSet` tipini ve eşzamanlılık modunu belirtmenize izin veren overload edilmiş imzalar vardır. Sınavda, yalnızca sonuçları sırayla işleyen varsayılan seçenekleri nasıl kullanacağınızı bilmeniz gerekir.

Java 17/JDBC editor note: Default `TYPE_FORWARD_ONLY`, cursor'ın yalnız ileri yönde hareketini açıklar; SQL row order garantisi vermez. `ORDER BY` bulunmayan bir query'nin row sırasına portable code güvenmemelidir.

Executing a PreparedStatement

English: Now that we have a PreparedStatement, we can run the SQL statement. The method for running SQL varies depending on what kind of SQL statement it is. Remember that you aren't expected to be able to read SQL, but you do need to know what the first keyword means.

Türkçe: Artık bir PreparedStatementımız olduğuna göre SQL statement'ını çalıştırabiliriz. Kullanılacak method, SQL statement'ının türüne göre değişir. SQL okuyabilmeniz beklenmediğini, ancak ilk anahtar kelimenin ne anlama geldiğini bilmeniz gerektiğini unutmayın.

Modifying Data with executeUpdate()

English: Let's start with statements that change the data in a table. Those are SQL statements that begin with DELETE, INSERT, or UPDATE. They typically use a method called executeUpdate(). The name is a little tricky because the SQL UPDATE statement is not the only statement that uses this method.

Türkçe: Table'daki veriyi değiştiren statement'larla başlayalım. Bunlar DELETE, INSERT veya UPDATE ile başlayan SQL statement'larıdır ve genellikle executeUpdate() method'unu kullanır. Ad biraz yanıltıcıdır; çünkü bu method'u yalnızca SQL UPDATE statement'ı kullanmaz.

English: The method takes the SQL statement to run as a parameter. It returns the number of rows that were inserted, deleted, or changed. Here's an example of all three update types:

Türkçe: Method, çalıştırılacak SQL statement'ını parameter olarak alır ve eklenen, silinen veya değiştirilen row sayısını döndürür. Üç update türünü gösteren örnek şöyledir:

```
10: var insertSql = "INSERT INTO exhibits VALUES(10, 'Deer', 3)";
11: var updateSql = "UPDATE exhibits SET name = ' ' " +
12:   "WHERE name = 'None'";
13: var deleteSql = "DELETE FROM exhibits WHERE id = 10";
14:
15: try (var ps = conn.prepareStatement(insertSql)) {
16:     int result = ps.executeUpdate();
17:     System.out.println(result); // 1
18: }
19:
20: try (var ps = conn.prepareStatement(updateSql)) {
21:     int result = ps.executeUpdate();
22:     System.out.println(result); // 0
23: }
```

```
24:
25: try (var ps = conn.prepareStatement(deleteSql)) {
26:     int result = ps.executeUpdate();
27:     System.out.println(result); // 1
28: }
```

English: For the exam, you don't need to read SQL. The question will tell you how many rows are affected if you need to know. Notice how each distinct SQL statement needs its own `prepareStatement()` call.

Türkçe: Sınav için SQL okumanıza gerek yoktur. Soru, bilmeniz gerekiyorsa kaç satırın etkilendiğini söyleyecektir. Her farklı SQL ifadesinin kendi `prepareStatement()` çağrısına nasıl ihtiyaç duyduğuna dikkat edin.

English: Line 15 creates the insert statement, and line 16 runs that statement to insert one row. The result is 1 because one row was affected. Line 20 creates the update statement, and line 21 checks the whole table for matching records to update. Since no records match, the result is 0. Line 25 creates the delete statement, and line 26 deletes the row created on line 16. Again, one row is affected, so the result is 1.

Türkçe: Satır 15, ekleme ifadesini oluşturur ve satır 16, bir satır eklemek için bu ifadeyi çalıştırır. Sonuç 1, çünkü bir satır etkilendi. Satır 20, güncelleme ifadesini oluşturur ve satır 21, güncellenecek kayıtları eşleştirmek için tüm tabloyu kontrol eder. Kayıt eşleşmesi olmadığından sonuç 0'dır. Satır 25, silme ifadesini oluşturur ve satır 26, satır 16'da oluşturulan satırı siler. Yine bir satır etkilenir, sonuç 1 olur.

Reading Data with `executeQuery()`

English: Next, let's look at a SQL statement that begins with SELECT. This time, we use the `executeQuery()` method.

Türkçe: Ardından, `SELECT` ile başlayan bir SQL ifadesine bakalım. Bu kez `executeQuery()` metodunu kullanıyoruz.

```
30: var sql = "SELECT * FROM exhibits";
31: try (var ps = conn.prepareStatement(sql);
32:     ResultSet rs = ps.executeQuery()) {
33:
34:     // work with rs
35: }
```

English: On line 31, we create a `PreparedStatement` for our SELECT query. On line 32, we run it. Since we are running a query to get a result, the return type is `ResultSet`. In the next section, we show you how to process the `ResultSet`.

Türkçe: 31. satırda `SELECT` query'miz için bir `PreparedStatement` oluşturur, 32. satırda çalıştırırız. Sonuç almak için query çalıştırdığımızdan return type `ResultSet` tir. Sonraki bölümde `ResultSet` in nasıl işleneceğini göstereceğiz.

Processing Data with `execute()`

English: There's a third method called `execute()` that can run either a query or an update. It returns a boolean so that we know whether there is a `ResultSet`. That way, we can call the proper method to get more detail. The pattern looks like this:

Türkçe: Query veya update çalıştırabilen üçüncü bir method, `execute()` dur. İlk sonucun `ResultSet` olup olmadığını bildiren bir `boolean` döndürür; böylece ayrıntıyı almak için doğru method çağrılabilir. Pattern şöyledir:

```
boolean isResultSet = ps.execute();
if (isResultSet) {
    try (ResultSet rs = ps.getResultSet()) {
        System.out.println("ran a query");
    }
} else {
    int result = ps.getUpdateCount();
    System.out.println("ran an update");
}
```

English: If the PreparedStatement refers to sql that is a SELECT, the boolean is true, and we can get the ResultSet. If it is not a SELECT, we can get the number of rows updated.

Türkçe: PreparedStatement, SELECT olan bir SQL'i gösteriyorsa boolean değer true olur ve ResultSet alınabilir. SELECT değilse update edilen row sayısı alınabilir.

Using the Correct Method

English: What do you think happens if we use the wrong method for a SQL statement? Let's take a look:

Türkçe: SQL ifadesi için yanlış metot kullanırsak ne olur sizce? Bir göz atalım:

```
var sql = "SELECT * FROM names";
try (var ps = conn.prepareStatement(sql)) {
    var result = ps.executeUpdate();
}
```

English: This throws a SQLException similar to the following:

Türkçe: Bu code, aşağıdakine benzer bir SQLException fırlatır:

```
Exception in thread "main" java.sql.SQLException:
statement does not generate a row count
```

English: We can't get a compiler error since the SQL is a String. We can get an exception, though, and we do. We also get a SQLException when using executeQuery() with SQL that changes the database.

Türkçe: SQL bir String olduğu için compiler error oluşmaz. Bunun yerine runtime'da exception fırlatılır. Veritabanını değiştiren bir SQL ile executeQuery() kullanıldığında da SQLException fırlatılır.

```
Exception in thread "main" java.sql.SQLException:
statement does not generate a result set
```

English: Again, we get an exception because the driver can't translate the query into the expected return type.

Türkçe: Yine exception oluşur; çünkü driver query'yi beklenen return type'a dönüştüremez.

Reviewing PreparedStatement Methods

English: To review, make sure that you know TABLE 15.3 and TABLE 15.4 well. TABLE 15.3 shows which SQL statements can be run by each of the three key methods on PreparedStatement. TABLE 15.4 shows what is returned by each method.

Türkçe: İncelemek için, TABLE 15.3 ve TABLE 15.4'ü iyi bildiğinizden emin olun. TABLE 15.3 hangi SQL ifadelerinin `PreparedStatement` üzerindeki üç anahtar metodun her biri tarafından çalıştırılabileceğini gösterir. TABLE 15.4, her bir metotla neyin iade edildiğini gösterir.

English: TABLE 15.3 · SQL runnable by each execute method.

Türkçe: TABLE 15.3 SQL her yürütme metoduyla çalıştırılabilir.

Method	DELETE	INSERT	SELECT	UPDATE
<code>ps.execute()</code>	Yes	Yes	Yes	Yes
<code>ps.executeQuery()</code>	No	No	Yes	No
<code>ps.executeUpdate()</code>	Yes	Yes	No	Yes

English: TABLE 15.4 · Return types of execute methods.

Türkçe: TABLE 15.4 Return types yürütme metotları.

Method	Return type	What is returned for SELECT	What is returned for DELETE / INSERT / UPDATE
<code>ps.execute()</code>	<code>boolean</code>	<code>true</code>	<code>false</code>
<code>ps.executeQuery()</code>	<code>ResultSet</code>	Rows and columns returned	n/a
<code>ps.executeUpdate()</code>	<code>int</code>	n/a	Number of rows added, changed, or removed

Java 17/JDBC contract: `execute()` does not report whether execution “succeeded” or classify the SQL text itself. It reports whether the first result is a `ResultSet`; `false` can also represent an update count or no result. Multiple-result statements require the related JDBC result-navigation APIs.

Working with Parameters

Dil çalışması: Bu başlıktaki kelimeler için [ünite sözlüğüne](#) bak.

English: Suppose our zoo acquires a new elephant and we want to register it in our names table. We’ve already learned enough to do this.

Türkçe: Hayvanat bahçemize yeni bir fil geldiğini ve onu `names` table'ına kaydetmek istediğimizi varsayalım. Bunu yapacak kadar bilgi öğrendik.

```
public static void register(Connection conn) throws SQLException {
    var sql = "INSERT INTO names VALUES(6, 1, 'Edith')";
    try (var ps = conn.prepareStatement(sql)) {
        ps.executeUpdate();
    }
}
```

English: However, everything is hard-coded. We want to be able to pass in the values as parameters. Luckily, a `PreparedStatement` allows us to set parameters. Instead of specifying the three values in the SQL, we can use a question mark (?). A bind variable is a placeholder that lets you specify the actual values at runtime. A bind variable is like a parameter, and you will see bind variables referenced as both variables and parameters. We can rewrite our SQL statement using bind variables.

Türkçe: Ancak her şey hard-coded'dır; değerleri parameter olarak geçirmek isteriz.

`PreparedStatement` parameter ayarlamamızı sağlar. SQL içinde üç değeri doğrudan yazmak yerine question mark (?) kullanabiliriz. Bind variable, gerçek değeri runtime'da belirtmemizi sağlayan bir placeholder'dır. Parameter'a benzediği için kaynaklarda bind variable'a hem variable hem de parameter dendiğini görebilirsiniz. SQL statement'ını bind variable'larla yeniden yazabiliriz.

```
String sql = "INSERT INTO names VALUES(?, ?, ?)";
```

English: Bind variables make the SQL easier to read since you no longer need to use quotes around String values in the SQL. Now we can pass the parameters to the method itself.

Türkçe: Bind variable'lar, SQL'deki `String` değerlerin çevresinde quotation mark kullanmanız gerekmediğinden SQL'in okunmasını kolaylaştırır. Şimdi parametreleri metodun kendisine aktarabiliriz.

```
14: public static void register(Connection conn, int key,
15:     int type, String name) throws SQLException {
16:
17:     String sql = "INSERT INTO names VALUES(?, ?, ?)";
18:
19:     try (PreparedStatement ps = conn.prepareStatement(sql)) {
20:         ps.setInt(1, key);
```

```
21:         ps.setString(3, name);
22:         ps.setInt(2, type);
23:         ps.executeUpdate();
24:     }
25: }
```

English: Line 19 creates a `PreparedStatement` using our SQL that contains three bind variables. Lines 20–22 set those variables. You can think of the bind variables as a list of parameters, where each is set in turn. Notice how the bind variables can be set in any order. Line 23 executes the query and runs the update.

Türkçe: 19. satır, üç bind variable içeren SQL ile bir `PreparedStatement` oluşturur. 20–22. satırlar bu variable'ları ayarlar. Bind variable'ları, her öğesi ayrı ayrı ayarlanan bir parameter listesi gibi düşünebilirsiniz. Herhangi bir sırada ayarlanabildiklerine dikkat edin. 23. satır query'yi çalıştırarak update'i gerçekleştirir.

English: Notice how the bind variables are counted starting with 1 rather than 0. This is really important, so we will repeat it.

Türkçe: Bind variable index'lerinin 0 yerine 1'den başlayarak sayıldığına dikkat edin. Bu gerçekten önemli, bu yüzden tekrarlayacağız.

English: Remember that JDBC starts counting columns with 1 rather than 0.

Türkçe: JDBC, column index'lerini 0'dan değil 1'den başlayarak sayar.

English: A common exam question tests that you know this!

Türkçe: Yaygın bir sınav sorusu, bu kuralı bilip bilmediğinizi ölçer.

English: In the previous example, we set the parameters out of order. That's perfectly fine. The rule is only that they are each set before the query is executed. Let's see what happens if you don't set all the bind variables.

Türkçe: Önceki örnekte parameter'ları farklı sırada ayarladık; bu tamamen geçerlidir. Tek kural, query çalıştırılmadan önce her birinin ayarlanmış olmasıdır. Tüm bind variable'ları ayarlamazsanız ne olacağına bakalım.

```
var sql = "INSERT INTO names VALUES(?,?,?)";
try (var ps = conn.prepareStatement(sql)) {
    ps.setInt(1, key);
    ps.setInt(2, type);
    //missing the set for parameter number 3
    ps.executeUpdate();
}
```

English: The code compiles, and you get a `SQLException`. The message may vary based on your database driver.

Türkçe: Code derlenir ve `SQLException` fırlatılır. Mesaj database driver'ınıza göre değişebilir.

```
Exception in thread "main" java.sql.SQLException: Parameter not set
```

English: What about if you try to set more values than you have as bind variables?

Türkçe: bind variables olarak sahip olduğunuzdan daha fazla değer belirlemeye çalışırsanız ne olur?

```
var sql = "INSERT INTO names VALUES(?,?)";
try (var ps = conn.prepareStatement(sql)) {
    ps.setInt(1, key);
    ps.setInt(2, type);
    ps.setString(3, name);
    ps.executeUpdate();
}
```

English: Again, you get a `SQLException`, this time with a different message. On HyperSQL, that message was as follows:

Türkçe: Yine, bu kez farklı bir mesajla `SQLException` elde edersiniz. HyperSQL'de bu mesaj şöyleydi:

```
Exception in thread "main" java.sql.SQLException:
row column count mismatch in statement [INSERT INTO names VALUES(?,?)]
```

English: TABLE 15.5 shows the methods you need to know for the exam to set bind variables. The ones that you need to know for the exam are easy to remember since they are called set followed by the name of the type you are setting. There are many others, like dates, that are out of scope for the exam.

Türkçe: TABLE 15.5, bind variable ayarlamak için sınavda bilmeniz gereken method'ları gösterir. Adları `set` ile başlayıp ayarlanan type'ın adıyla devam ettiği için kolay hatırlanır. Date ile ilgili olanlar gibi başka birçok setter sınav kapsamı dışındadır.

English: TABLE 15.5 · PreparedStatement methods.

Türkçe: TABLE 15.5 `PreparedStatement` metotları.

Method	Java parameter type	Example database type
<code>setBoolean</code>	<code>boolean</code>	<code>BOOLEAN</code>
<code>setDouble</code>	<code>double</code>	<code>DOUBLE</code>
<code>setInt</code>	<code>int</code>	<code>INTEGER</code>
<code>setLong</code>	<code>long</code>	<code>BIGINT</code>
<code>setNull</code>	<code>int</code>	Any type
<code>setObject</code>	<code>Object</code>	Any type
<code>setString</code>	<code>String</code>	<code>CHAR</code> , <code>VARCHAR</code>

English: The first column shows the method name, and the second column shows the type that Java uses. The third column shows the type name that could be in the database. There is some variation by databases, so check your specific database documentation. You need to know only the first two columns for the exam.

Türkçe: İlk column method adını, ikinci column Java'nın kullandığı type'ı, üçüncü column ise veritabanında bulunabilecek type adını gösterir. Veritabanları arasında farklılık olabildiğinden kullandığınız veritabanının documentation'ına bakın. Sınav için yalnızca ilk iki column'u bilmeniz gerekir.

English: The `setNull()` method takes an `int` parameter representing the column type in the database. You do not need to know these types. Notice that the `setObject()` method works with any Java type. If you pass a primitive, it will be autoboxed into a wrapper type. That means we can rewrite our example as follows:

Türkçe: `setNull()` method'u veritabanındaki column type'ı temsil eden bir `int` parameter alır; bu type'ları bilmeniz gerekmez. `setObject()` in her Java type'ıyla çalışabildiğine dikkat edin. Primitive bir değer geçirirseniz wrapper type'a autobox edilir. Bu nedenle örneği şöyle yeniden yazabiliriz:

Java 17/JDBC API ayrıntısı: Temel imza `setNull(int parameterIndex, int sqlType)` biçimindedir; yani table'daki `int`, yalnız SQL type argument'ını gösterir ve method ayrıca 1-based parameter index'i alır. `setObject()` geniş bir Java type alanını kabul etse de her Java-to-SQL conversion'ın her driver tarafından kabul edilmesi garanti değildir.

```
String sql = "INSERT INTO names VALUES(?,?,?)";
try (PreparedStatement ps = conn.prepareStatement(sql)) {
    ps.setObject(1, key);
    ps.setObject(2, type);
    ps.setObject(3, name);
    ps.executeUpdate();
}
```

English: Java will handle the type conversion for you. It is still better to call the more specific setter methods since that will give you a compile-time error if you pass the wrong type instead of a runtime error.

Türkçe: Java sizin için tür dönüştürme işlemini halledecektir. Yine de daha spesifik ayarlayıcı metotlarını çağırmak daha iyidir, çünkü bu, çalışma zamanı hatası yerine yanlış tip geçerseniz size bir derleme zamanı hatası verecektir.

Updating Multiple Records

English: Suppose we get two new elephants and want to add both. We can use the same `PreparedStatement` object.

Türkçe: Diyelim ki iki yeni fil aldık ve ikisini de eklemek istiyoruz. Aynı `PreparedStatement` nesnesini kullanabiliriz.

```
var sql = "INSERT INTO names VALUES(?,?,?)";
try (var ps = conn.prepareStatement(sql)) {
    ps.setInt(1, 20);
    ps.setInt(2, 1);
    ps.setString(3, "Ester");
    ps.executeUpdate();
    ps.setInt(1, 21);
    ps.setString(3, "Elias");
    ps.executeUpdate();
}
```

English: Note that we set all three parameters when adding Ester but only two for Elias. The `PreparedStatement` is smart enough to remember the parameters that were already set and retain them. You only have to set the ones that are different.

Türkçe: Ester'i eklerken üç parametreye de değer atadığımıza, Elias içinse yalnız ikisini değiştirdiğimize dikkat edin. `PreparedStatement`, daha önce atanmış parametre değerlerini hatırlar ve korur. Yalnız değişen değerlere yeniden atama yapmanız gerekir.

Batching Statements

English: JDBC supports batching so you can run multiple statements in fewer trips to the database. Often the database is located on a different machine than the Java code runs on. Saving trips to the database saves time because network calls can be expensive. For example, if you need to insert 1,000 records into the database, inserting them as a single network call (as opposed to 1,000 network calls) is usually a lot faster.

Türkçe: JDBC batching'i destekler; böylece veritabanına daha az gidiş-gelişle birden çok statement çalıştırabilirsiniz. Veritabanı çoğu zaman Java code'unun çalıştığı makineden farklı bir makinededir. Network call'lar maliyetli olabildiğinden database'e yapılan round trip sayısını azaltmak zaman kazandırır. Örneğin 1.000 kaydı 1.000 ayrı network call yerine tek bir network call ile eklemek genellikle çok daha hızlıdır.

English: You don't need to know the `addBatch()` and `executeBatch()` methods for the exam, but they are useful in practice.

Türkçe: Sınav için `addBatch()` ve `executeBatch()` metotlarını bilmenize gerek yoktur, ancak uygulamada yararlıdırlar.

```
public static void register(Connection conn, int firstKey,
int type, String... names) throws SQLException {
    var sql = "INSERT INTO names VALUES (?, ?, ?)";
    var nextIndex = firstKey;
```

```
    try (var ps = conn.prepareStatement(sql)) {
        ps.setInt(2, type);
        for(var name: names) {
            ps.setInt(1, nextIndex);
            ps.setString(3, name);
            ps.addBatch();
            nextIndex++;
        }
        int[] result = ps.executeBatch();
        System.out.println(Arrays.toString(result));
    }
}
```

English: Now we call this method with two names:

Türkçe: Şimdi bu method'u iki name ile çağırıyoruz:

```
register(conn, 100, 1, "Elias", "Ester");
```

English: The output shows that the array has two elements since there are two different items in the batch. Each added one row in the database.

Türkçe: Çıktı, dizide iki farklı öge olduğundan dizinin iki elemanı olduğunu gösterir. Her biri veritabanına bir satır ekledi.

```
[1, 1]
```

English: You can use batching to break up large operations, such as inserting 10 million records in groups of 100. In practice, it takes a bit of work to determine an appropriate batch size, but the performance of using batch is normally far better than inserting one row at a time (or all ten million at once).

Türkçe: On milyon kaydı 100'lük gruplar halinde eklemek gibi büyük işlemleri bölmek için batching kullanabilirsiniz. Uygun batch size'ı belirlemek pratikte biraz çalışma gerektirir; fakat batching performansı genellikle row'ları tek tek veya on milyonun tamamını bir defada eklemekten çok daha iyidir.

Getting Data from a ResultSet

English: A database isn't useful if you can't get your data. We start by showing you how to go through a ResultSet. Then we go through the different methods to get columns by type.

Türkçe: Veri tabanı, verilerinizi alamıyorsanız kullanışlı değildir. **ResultSet** üzerinden nasıl geçileceğini göstererek başlıyoruz. Daha sonra sütunları tipe göre almak için farklı metotlardan geçeriz.

Reading a ResultSet

English: When working with a ResultSet, most of the time, you will write a loop to look at each row. The code looks like this:

Türkçe: Bir **ResultSet** ile çalışırken, çoğu zaman, her satıra bakmak için bir döngü yazacaksınız. Kod şu şekilde görünüyor:

```
20: String sql = "SELECT id, name FROM exhibits";
21: var idToNameMap = new HashMap<Integer, String>();
22:
```

```
23: try (var ps = conn.prepareStatement(sql);
24:      ResultSet rs = ps.executeQuery()) {
25:
26:     while (rs.next()) {
27:         int id = rs.getInt("id");
28:         String name = rs.getString("name");
29:         idToNameMap.put(id, name);
30:     }
31:     System.out.println(idToNameMap);
32: }
```

English: It outputs this:

Türkçe: Şu çıktıyı verir:

```
{1=African Elephant, 2=Zebra}
```

English: There are a few things to notice here. First, we use the `executeQuery()` method on line 24, since we want to have a `ResultSet` returned. On line 26, we loop through the results. Each time through the loop represents one row in the `ResultSet`. Lines 27 and 28 show you the best way to get the columns for a given row.

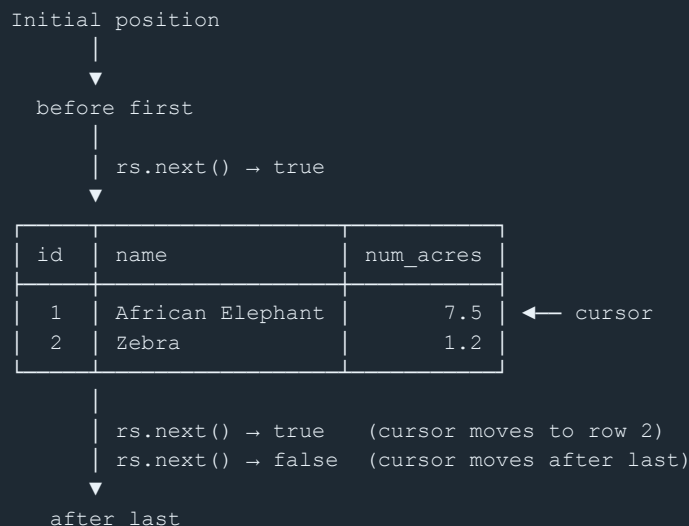
Türkçe: Burada birkaç noktaya dikkat edin. `ResultSet` dönmesini istediğimiz için 24. satırda `executeQuery()` çağırıyoruz. 26. satırda sonuçlar üzerinde loop'a giriyoruz; her iteration, `ResultSet`teki bir row'u temsil eder. 27 ve 28. satırlar, belirli bir row'un column'larını almanın en iyi yolunu gösterir.

English: A `ResultSet` has a cursor, which points to the current location in the data. FIGURE 15.5 shows the position as we loop through.

Türkçe: Bir `ResultSet`, data içindeki mevcut konumu gösteren cursor'a sahiptir. FIGURE 15.5, loop sırasında cursor'ın konumlarını gösterir.

English: FIGURE 15.5 · The `ResultSet` cursor.

Türkçe: FIGURE 15.5 `ResultSet` imleci.



English: At line 24, the cursor starts by pointing to the location before the first row in the `ResultSet`. On the first loop iteration, `rs.next()` returns `true`, and the cursor moves to point to the first row of data. On the second loop iteration, `rs.next()` returns `true` again, and the cursor moves to point to the second row of data. The next call to `rs.next()` returns `false`. The cursor advances past the end of the data. The `false` signifies that there is no more data available to get.

Türkçe: 24. satırda cursor, `ResultSet`teki ilk row'dan önceki konumu gösterir. İlk loop iteration'ında `rs.next()` `true` döndürür ve cursor ilk data row'una ilerler. İkinci iteration'da yine `true` döndürür ve cursor ikinci row'a ilerler. Sonraki `rs.next()` çağırısı `false` döndürerek cursor'ı verinin sonrasına taşır; `false`, alınabilecek başka data kalmadığını gösterir.

English: We did say the “best way” to get data was with column names. There is another way to access the columns. You can use an index, counting from 1 instead of a column name.

Türkçe: Verileri elde etmenin en iyi yolunun sütun isimleri olduğunu söyledik. Sütunlara ulaşmanın başka bir yolu daha var. Bir sütun adı yerine 1'den sayarak bir indeks kullanabilirsiniz.

```
int id = rs.getInt(1);
String name = rs.getString(2);
```

English: Now you can see the column positions. Notice how the columns are counted starting with 1 rather than 0. Just like with a PreparedStatement, JDBC starts counting at 1 in a ResultSet.

Türkçe: Şimdi sütun pozisyonlarını görebilirsiniz. Sütunların 0 yerine 1 ile nasıl sayıldığına dikkat edin. Tıpkı bir PreparedStatement ile olduğu gibi, JDBC bir ResultSet içinde 1'den saymaya başlar.

English: The column name is better because it is clearer what is going on when reading the code. It also allows you to change the SQL to reorder the columns.

Türkçe: Sütun adı daha iyidir, çünkü kodu okurken neler olduğu daha nettir. Ayrıca, sütunları yeniden sıralamak için SQL'i değiştirmenize izin verir.

English: On the exam, either you will be told the names of the columns in a table, or you can assume that they are correct. Similarly, you can assume that all SQL is correct.

Türkçe: Sınavda, ya bir tablodaki sütunların isimleri size söylenecek ya da doğru olduklarını varsayabilirsiniz. Benzer şekilde, tüm SQL doğru olduğunu varsayabilirsiniz.

English: Sometimes you want to get only one row from the table. Maybe you need only one piece of data. Or maybe the SQL is just returning the number of rows in the table. When you want only one row, you use an if statement rather than a while loop.

Türkçe: Bazen table'dan yalnızca bir row almak istersiniz. Tek bir data parçasına ihtiyacınız olabilir veya SQL yalnızca table'daki row sayısını döndürüyor olabilir. Tek row istediğinizde while loop yerine if statement kullanırsınız.

```
var sql = "SELECT count(*) FROM exhibits";
try (var ps = conn.prepareStatement(sql);
    var rs = ps.executeQuery()) {
    if (rs.next()) {
        int count = rs.getInt(1);
        System.out.println(count);
    }
}
```

English: It is important to check that `rs.next()` returns true before trying to call a getter on the `ResultSet`. If a query didn't return any rows, it would throw a `SQLException`, so the if statement checks that it is safe to call. Alternatively, you can use the column name.

Türkçe: `ResultSet` üzerinde getter çağırmadan önce `rs.next()` in `true` döndürdüğünü kontrol etmek önemlidir. Query hiç row döndürmediyse getter çağırısı `SQLException` fırlatır; dolayısıyla if statement çağırının güvenli olup olmadığını denetler. Alternatif olarak column adını kullanabilirsiniz.

```
var count = rs.getInt("count");
```

English: Let's try to read a column that does not exist.

Türkçe: Var olmayan bir sütunu okumaya çalışalım.

```
var sql = "SELECT count(*) AS count FROM exhibits";
try (var ps = conn.prepareStatement(sql);
    var rs = ps.executeQuery()) {
    if (rs.next()) {
        var count = rs.getInt("total");
        System.out.println(count);
    }
}
```

English: This throws a `SQLException` with a message like this:

Türkçe: Bu şöyle bir mesajla `SQLException` atar:

```
Exception in thread "main" java.sql.SQLException: Column not found: total
```

English: Attempting to access a column name or index that does not exist throws a `SQLException`, as does getting data from a `ResultSet` when it isn't pointing at a valid row. You need to be able to recognize such code. Here are a few examples to watch out for. Do you see what is wrong when no rows match?

Türkçe: Var olmayan bir sütun adına veya indekse erişmeye çalışmak, geçerli bir satırı işaret etmediğinde bir `ResultSet`ten veri almak gibi `SQLException` fırlatır. Böyle bir code'u tanımanız gerekir. İşte dikkat edilmesi gereken birkaç örnek. Satır eşleşmediği zaman neyin yanlış olduğunu görüyor musun?

```
var sql = "SELECT * FROM exhibits where name='Not in table'";
try (var ps = conn.prepareStatement(sql);
    var rs = ps.executeQuery()) {
    rs.next();
    rs.getInt(1); // SQLException
}
```

English: Calling `rs.next()` works. It returns `false`. However, calling a getter afterward throws a `SQLException` because the result set cursor does not point to a valid position. If a match were returned, this code would have worked. Do you see what is wrong with the following?

Türkçe: `rs.next()` çağırısı çalışır ve `false` döndürür. Ancak ardından getter çağırmak `SQLException` fırlatır; çünkü `ResultSet` cursor'ı geçerli bir row'u göstermez. Eşleşen bir row dönseydi bu code çalışırdı. Aşağıdaki code'da neyin yanlış olduğunu görebiliyor musunuz?

```
var sql = "SELECT count(*) FROM exhibits";
try (var ps = conn.prepareStatement(sql);
    var rs = ps.executeQuery()) {
    rs.getInt(1); // SQLException
}
```

English: Not calling `rs.next()` at all is a problem. The result set cursor is still pointing to a location before the first row, so the getter has nothing to point to.

Türkçe: `rs.next()` i hiç çağırmamak sorundur. `ResultSet` cursor'ı hâlâ ilk row'dan önceki konumu gösterdiği için getter'ın okuyabileceği bir row yoktur.

English: To sum up this section, it is important to remember the following:

Türkçe: Bu bölümü özetlemek gerekirse, aşağıdakileri hatırlamak önemlidir:

English: Always use an if statement or while loop when calling `rs.next()`.

Türkçe: `rs.next()` çağırırken her zaman bir if ifadesi veya while döngüsü kullanın.

English: Column indexes begin with 1.

Türkçe: Sütun indeksleri 1 ile başlar.

Getting Data for a Column

English: There are lots of get methods on the `ResultSet` interface. TABLE 15.6 shows the get methods that you need to know. These are the getter equivalents of the setters in TABLE 15.5.

Türkçe: `ResultSet` interface'inde birçok metot vardır. TABLE 15.6, bilmeniz gereken metotları gösterir. Bunlar TABLE 15.5'teki setterlerin getter eşdeğerleridir.

English: TABLE 15.6 · ResultSet get methods.

Türkçe: TABLE 15.6 · `ResultSet` get method'ları.

Method	Return type
<code>getBoolean</code>	<code>boolean</code>
<code>getDouble</code>	<code>double</code>
<code>getInt</code>	<code>int</code>
<code>getLong</code>	<code>long</code>
<code>getObject</code>	<code>Object</code>
<code>getString</code>	<code>String</code>

English: You might notice that not all of the primitive types are in TABLE 15.6. There are `getBytes()` and `getFloat()` methods, but you don't need to know about them for the exam. There is no `getChar()` method. Luckily, you don't need to remember this. The exam will not try to trick you by using a get method name that doesn't exist for JDBC. Isn't that nice of the exam creators?

Türkçe: İlkel türlerin hepsinin TABLE 15.6'da olmadığını fark edebilirsiniz. `getBytes()` ve `getFloat()` metotları vardır, ancak sınav için bunları bilmeniz gerekmez. `getChar()` metodu yoktur. Neyse ki bunu hatırlamanıza gerek yok. Sınav, JDBC için mevcut olmayan bir get metodu adı kullanarak sizi kandırmaya çalışmayacaktır. Bu sınav yaratıcıları için hoş değil mi?

English: The `getObject()` method can return any type. For a primitive, it uses the wrapper class. Let's look at the following example:

Türkçe: `getObject()` herhangi bir type döndürebilir; primitive değerler için ilgili wrapper class kullanılır. Aşağıdaki örneğe bakalım:

```

16: var sql = "SELECT id, name FROM exhibits";
17: try (var ps = conn.prepareStatement(sql);
18:     var rs = ps.executeQuery()) {
19:
20:     while (rs.next()) {
21:         Object idField = rs.getObject("id");
22:         Object nameField = rs.getObject("name");
23:         if (idField instanceof Integer id)
24:             System.out.println(id);
25:         if (nameField instanceof String name)
26:             System.out.println(name);
27:     }
28: }

```

English: Lines 21 and 22 get the column as whatever type of Object is most appropriate. Lines 23–26 use pattern matching to get the actual types. You probably won't use getObject() when writing code for a job, but it is good to know about it for the exam.

Türkçe: 21 ve 22. satırlar column'u en uygun `Object` type'ıyla alır. 23–26. satırlar gerçek type'ları elde etmek için pattern matching kullanır. Profesyonel code yazarken `getObject()` ı muhtemelen kullanmazsınız; ancak sınav için bilmeniz yararlıdır.

Using Bind Variables

English: We've been creating the PreparedStatement and ResultSet in the same try-with-resources statement. This doesn't work if you have bind variables because they need to be set in between. Luckily, we can nest try-with-resources to handle this. This code prints out the ID for any exhibits matching a given name:

Türkçe: `PreparedStatement` ve `ResultSet` i aynı try-with-resources statement'ta oluşturuyoruz. Bu, bind variables varsa işe yaramaz, çünkü aralarında ayarlanması gerekir. Neyse ki, bunu halletmek için try-with-resources yuvalayabiliriz. Bu kod, belirli bir isimle eşleşen herhangi bir serginin kimliğini yazdırır:

```
30: var sql = "SELECT id FROM exhibits WHERE name = ?";
31:
32: try (var ps = conn.prepareStatement(sql)) {
33:     ps.setString(1, "Zebra");
34:
35:     try (var rs = ps.executeQuery()) {
36:         while (rs.next()) {
37:             int id = rs.getInt("id");
38:             System.out.println(id);
39:         }
40:     }
41: }
```

English: Pay attention to the flow here. First we create the PreparedStatement on line 32. Then we set the bind variable on line 33. It is only after these are both done that we have a nested try-with-resources on line 35 to create the ResultSet.

Türkçe: Buradaki akışa dikkat edin. Önce 32. satırda `PreparedStatement` oluşturur, ardından 33. satırda bind variable'ı ayarlarız. Ancak bu iki adım tamamlandıktan sonra 35. satırda `ResultSet` oluşturan nested try-with-resources block'una gireriz.

Calling a CallableStatement

English: In some situations, it is useful to store SQL queries in the database instead of packaging them with the Java code. This is particularly useful when there are many complex queries. A stored procedure is code that is compiled in advance and stored in the database. Stored procedures are commonly written in a database-specific variant of SQL, which varies among database software providers.

Türkçe: Bazı durumlarda, SQL sorgularını Java koduyla paketlemek yerine veritabanında saklamak yararlıdır. Bu özellikle birçok karmaşık sorgu olduğunda yararlıdır. stored procedure önceden derlenen ve veritabanında saklanan koddur. Stored procedures genellikle veritabanı yazılım sağlayıcıları arasında değişen SQL veri tabanına özgü bir varyantta yazılmıştır.

English: Using a stored procedure reduces network round trips. It also allows database experts to own that part of the code. However, stored procedures are database-specific and introduce complexity into maintaining your application. On the exam, you need to know how to call a stored procedure but not decide when to use one.

Türkçe: Stored procedure kullanmak network round trip sayısını azaltır ve code'un bu kısmının database uzmanlarınca yönetilmesini sağlar. Ancak stored procedure'ler veritabanına özgüdür ve uygulama bakımını karmaşıktırır. Sınavda stored procedure'ün nasıl çağrıldığını bilmeniz gerekir; ne zaman kullanılacağına karar vermeniz beklenmez.

English: You don't need to know how to read or write a stored procedure for the exam. Therefore, we have not included any in the book. They are in the code from setting up the sample database if you are curious.

Türkçe: Sınav için stored procedure okumayı veya yazmayı bilmeniz gerekmez; bu nedenle kitapta procedure tanımlarına yer verilmemiştir. Merak ederseniz bu tanımları örnek veritabanını kuran code içinde bulabilirsiniz.

English: You do not need to learn anything database-specific for the exam. Since studying stored procedures can be quite complicated, we recommend limiting your studying on CallableStatement to what is in this book.

Türkçe: Sınav için veritabanına özel bir şey öğrenmek zorunda değilsiniz. stored procedures üzerinde çalışmak oldukça karmaşık olabileceğinden, `CallableStatement` üzerinde çalışmanızı bu kitapta bulunanlarla sınırlamanızı tavsiye ederiz.

English: We will be using four stored procedures in this section. TABLE 15.7 summarizes what you need to know about them. In the real world, none of these would be good implementations since they aren't complex enough to warrant being stored procedures. As you can see in the table, stored procedures allow parameters to be for input only, output only, or both.

Türkçe: Bu bölümde dört stored procedure kullanacağız. TABLE 15.7, bunlar hakkında bilmeniz gerekenleri özetler. Gerçek dünyada hiçbir stored procedure olmayı gerektirecek kadar karmaşık olmadığı için iyi bir implementation sayılmaz. Table'da görüldüğü gibi stored procedure parameter'ları yalnız input, yalnız output veya her ikisi için kullanılabilir.

English: TABLE 15.7 · Sample stored procedures.

Türkçe: TABLE 15.7 Örnek stored procedures.

Name	Parameter name	Parameter type	Description
<code>read_e_names()</code>	n/a	n/a	Returns all rows in <code>names</code> whose name begins with e or E
<code>read_names_by_letter()</code>	<code>prefix</code>	IN	Returns all rows in <code>names</code> whose name begins with the specified parameter, case-insensitively
<code>magic_number()</code>	<code>num</code>	OUT	Returns the number 42
<code>double_number()</code>	<code>num</code>	INOUT	Multiplies the parameter by two and returns that number

English: In the next four sections, we look at how to call each of these stored procedures.

Türkçe: Sonraki dört bölümde bu stored procedure'lerin her birini nasıl çağıracağımızı inceleyeceğiz.

Calling a Procedure without Parameters

English: Our `read_e_names()` stored procedure doesn't take any parameters. It does return a `ResultSet`. Since we worked with a `ResultSet` in the `PreparedStatement` section, here we can focus on how the stored procedure is called.

Türkçe: `read_e_names()` stored procedure'ü parameter almaz ve bir `ResultSet` döndürür. `PreparedStatement` bölümünde `ResultSet` ile çalıştığımız için burada stored procedure'ün nasıl çağırıldığına odaklanabiliriz.

```
12: String sql = "{call read_e_names()}";
13: try (CallableStatement cs = conn.prepareCall(sql);
14:     ResultSet rs = cs.executeQuery()) {
15:
16:     while (rs.next()) {
17:         System.out.println(rs.getString(3));
18:     }
19: }
```

English: Line 12 introduces a new bit of syntax. A stored procedure is called by putting the word `call` and the procedure name in braces (`{}`). Line 13 creates a `CallableStatement` object. When we created a `PreparedStatement`, we used the `prepareStatement()` method. Here, we use the `prepareCall()` method instead.

Türkçe: 12. satır yeni bir syntax gösterir: stored procedure, `call` keyword'ü ve procedure adı braces (`{}`) içine yazılarak çağırılır. 13. satır bir `CallableStatement` object'i oluşturur. `PreparedStatement` oluştururken `prepareStatement()` kullanmıştık; burada bunun yerine `prepareCall()` kullanıyoruz.

English: Lines 14–18 should look familiar. They are the standard logic we have been using to get a `ResultSet` and loop through it. This stored procedure returns the underlying table, so the columns are the same.

Türkçe: 14–18. satırlar tanıdık gelmelidir. Bunlar `ResultSet` alıp üzerinde loop'a girmek için kullandığımız standart yapıdır. Stored procedure alttaki table'ı döndürdüğü için column'lar aynıdır.

Passing an IN Parameter

English: A stored procedure that always returns the same thing is only somewhat useful. We've created a new version of that stored procedure that is more generic. The `read_names_by_letter()` stored procedure takes a parameter for the prefix or first letter of the stored procedure. An IN parameter is used for input.

Türkçe: Her zaman aynı şeyi döndüren bir stored procedure sınırlı ölçüde kullanışlıdır. Bu procedure'ün daha generic bir sürümünü oluşturduk. `read_names_by_letter()` stored procedure'ü prefix, yani adın ilk harfi için bir parameter alır. Input için `IN` parameter kullanılır.

English: There are two differences in calling it compared to our previous stored procedure.

Türkçe: Önceki stored procedure'e göre çağrı biçiminde iki fark vardır.

```
25: var sql = "{call read_names_by_letter(?)}";
26: try (var cs = conn.prepareCall(sql)) {
27:     cs.setString("prefix", "Z");
28:
29:     try (var rs = cs.executeQuery()) {
30:         while (rs.next()) {
31:             System.out.println(rs.getString(3));
32:         }
33:     }
34: }
```

English: On line 25, we have to pass a ? to show we have a parameter. This should be familiar from bind variables with a `PreparedStatement`.

Türkçe: 25. satırda bir parameter bulunduğunu göstermek için question mark (?) geçiririz. `PreparedStatement` bind variable'larından bu kullanım tanıdık gelmelidir.

English: On line 27, we set the value of that parameter. Unlike with `PreparedStatement`, we can use either the parameter number (starting with 1) or the parameter name. That means these two statements are equivalent:

Türkçe: 27. satırda parameter'ın değerini ayarlarız. `PreparedStatement`tan farklı olarak 1'den başlayan parameter index'ini veya parameter name'i kullanabiliriz. Bu nedenle aşağıdaki iki statement eşdeğerdir:

```
cs.setString(1, "Z");
cs.setString("prefix", "Z");
```

Returning an OUT Parameter

English: In our previous examples, we returned a `ResultSet`. Some stored procedures return other information. Luckily, stored procedures can have OUT parameters for output. The `magic_number()` stored procedure sets its OUT parameter to 42. There are a few differences here:

Türkçe: Önceki örneklerde `ResultSet` döndürdük; bazı stored procedure'ler başka bilgiler döndürür. Bunun için OUT parameter kullanılabilir. `magic_number()` stored procedure'ü kendi OUT parameter'ini 42 olarak ayarlar. Burada birkaç fark vardır:

```
40: var sql = "{?= call magic_number(?) }";
41: try (var cs = conn.prepareStatement(sql)) {
```

```
42:     cs.registerOutParameter(1, Types.INTEGER);
43:     cs.execute();
44:     System.out.println(cs.getInt("num"));
45: }
```

English: On line 40, we include two special characters (`?=`) to specify that the stored procedure has an output value. This is optional since we have the OUT parameter, but it does aid in readability.

Türkçe: 40. satırda stored procedure'ün output value'ya sahip olduğunu belirtmek için `?=` karakterlerini ekledik. OUT parameter bulunduğu için kaynağa göre bu optional'dır, ancak okunabilirliğe yardımcı olur.

English: On line 42, we register the OUT parameter. This is important. It allows JDBC to retrieve the value on line 44. Remember to always call `registerOutParameter()` for each OUT or INOUT parameter (which we cover next).

Türkçe: 42. satırda OUT parameter'ı register ederiz. Bu adım JDBC'nin 44. satırdaki değeri alabilmesi için gereklidir. Her OUT veya INOUT parameter için `registerOutParameter()` çağırmaı unutmayın (bir sonraki bölümde INOUT ele alınır).

English: On line 43, we call `execute()` instead of `executeQuery()` since we are not returning a `ResultSet` from our stored procedure.

Türkçe: Stored procedure'ümüz `ResultSet` döndürmediği için 43. satırda `executeQuery()` yerine `execute()` çağırıyoruz.

Java 17/JDBC editor note: Source example'ında `{?= call magic_number(?)}` iki placeholder içerirken yalnız position 1 register edilir. JDBC escape syntax'ta leading `?=` function return value'yu, parentheses içindeki `?` ise ayrı procedure/function argument'ını temsil eder. Generic bir OUT parameter'ı ifade etmek için `?=` eklenmez. Exact callable signature ve accepted syntax database/driver contract'ıyla eşleşmelidir.

Database-Specific Behavior

English: Some databases are lenient about certain things this chapter says are required. For example, some databases allow you to omit the following:

Türkçe: Bazı veritabanları, chapter'ın zorunlu saydığı kimi öğeler konusunda esnektir. Örneğin aşağıdakilerin atlanmasına izin verebilir:

English: Braces ({})

Türkçe: Braces ({})

English: Bind variable (?) if it is an OUT parameter

Türkçe: OUT parameter söz konusuysa bind variable (?)

English: Call to registerOutParameter()

Türkçe: registerOutParameter() çağırısı

English: For the exam, you need to answer according to the full requirements, which are described in this book. For example, you should answer exam questions as if braces are required.

Türkçe: Sınavda bu kitapta açıklanan tüm gereksinimlere göre cevap vermelisiniz. Örneğin braces ({}) zorunluymuş gibi değerlendirme yapmalısınız.

Working with an INOUT Parameter

English: Finally, it is possible to use the same parameter for both input and output. As you read this code, see whether you can spot which lines are required for the IN part and which are required for the OUT part:

Türkçe: Son olarak aynı parameter hem input hem output için kullanılabilir. Code'u okurken IN kısmı için hangi satırların, OUT kısmı için hangilerinin gerekli olduğunu tespit edip edemeyeceğinize bakın:

```
50: var sql = "{call double_number(?)}";
51: try (var cs = conn.prepareStatement(sql)) {
52:     cs.setInt(1, 8);
53:     cs.registerOutParameter(1, Types.INTEGER);
54:     cs.execute();
55:     System.out.println(cs.getInt("num"));
56: }
```

English: For an IN parameter, line 52 is required since it sets the value. For an OUT parameter, line 53 is required to register it. Line 54 uses execute() again because we are not returning a ResultSet.

Türkçe: IN parameter için değeri ayarlayan 52. satır gerekir. OUT parameter için onu register eden 53. satır gerekir. ResultSet döndürmediğimiz için 54. satırda yine execute() kullanılır.

English: Remember that an INOUT parameter acts as both an IN parameter and an OUT parameter, so it has all the requirements of both.

Türkçe: Bir **INOUT** parameter'ın hem **IN** hem **OUT** parameter gibi davrandığını unutmayın, bu nedenle her ikisinin de tüm gereksinimlerine sahiptir.

Comparing Callable Statement Parameters

English: TABLE 15.8 reviews the different types of parameters. You need to know this well for the exam.

Türkçe: TABLE 15.8 farklı parametre türlerini gözden geçirir. Sınav için bunu iyi bilmeniz gerekiyor.

English: TABLE 15.8 · Stored procedure parameter types.

Türkçe: TABLE 15.8 Stored procedure parametre türleri.

Rule	IN	OUT	INOUT
Used for input	Yes	No	Yes
Used for output	No	Yes	Yes
Must set parameter value	Yes	No	Yes
Must call <code>registerOutParameter()</code>	No	Yes	Yes
Can include <code>?</code>	No	Yes	Yes

Java 17/JDBC contract: JDBC escape syntax'taki leading `?`, bir function'ın return placeholder'ıdır. Generic bir procedure **OUT/INOUT** parameter'ının kendisiyle aynı kavram değildir. **OUT** ve **INOUT** parameter'lar kendi positions'ında `registerOutParameter()` ile register edilir.

Using Additional Options

English: So far, we've been creating PreparedStatement and CallableStatement with the default options. Both support ResultSet type and concurrency options. Not all options are available on all databases. Luckily, you just have to be able to recognize them as valid on the exam.

Türkçe: Şimdiye kadar **PreparedStatement** ve **CallableStatement**ı default option'larla oluşturduk. Her ikisi de **ResultSet** type ve concurrency option'larını destekler. Tüm seçenekler tüm veritabanlarında mevcut değildir. Neyse ki, sadece onları sınavda geçerli olarak tanıyabilmeniz gerekir.

English: There are three ResultSet integer type values:

Türkçe: Üç **ResultSet** integer type değeri vardır:

English: `ResultSet.TYPE_FORWARD_ONLY`: Can go through the `ResultSet` only one row at a time
`ResultSet.TYPE_SCROLL_INSENSITIVE`: Can go through the `ResultSet` in any order but will not see changes made to the underlying database table
`ResultSet.TYPE_SCROLL_SENSITIVE`: Can go through the `ResultSet` in any order and will see changes made to the underlying database table
There are two `ResultSet` integer concurrency mode values:

Türkçe: `ResultSet.TYPE_FORWARD_ONLY`: `ResultSet` üzerinde yalnızca ileri yönde, row row ilerler.
`ResultSet.TYPE_SCROLL_INSENSITIVE`: `ResultSet` üzerinde herhangi bir sırada ilerler, ancak altta yatan veritabanı tablosunda yapılan değişiklikleri görmez
`ResultSet.TYPE_SCROLL_SENSITIVE`: `ResultSet` üzerinde herhangi bir sırada ilerler ve altta yatan veritabanı tablosunda yapılan değişiklikleri görür
İki `ResultSet` tamsayı eşzamanlılık modu değeri vardır:

English: `ResultSet.CONCUR_READ_ONLY`: The `ResultSet` cannot be updated.

Türkçe: `ResultSet.CONCUR_READ_ONLY`: `ResultSet` güncellenemez.

English: `ResultSet.CONCUR_UPDATABLE`: The `ResultSet` can be updated.

Türkçe: `ResultSet.CONCUR_UPDATABLE`: `ResultSet` güncellenebilir.

English: These options are integer values, not enum values, which means you pass both as additional parameters after the SQL.

Türkçe: Bu seçenekler, enum değerleri değil tamsayı değerleridir, yani SQL'dan sonra her ikisini de ek parametreler olarak geçersiniz.

```
conn.prepareCall(sql, ResultSet.TYPE_FORWARD_ONLY,
ResultSet.CONCUR_READ_ONLY);
conn.prepareStatement(sql, ResultSet.TYPE_SCROLL_INSENSITIVE,
ResultSet.CONCUR_UPDATABLE);
```

English: If you see these options on the exam, pay attention to how they are used.

Türkçe: Sınavda bu seçenekleri görürseniz, nasıl kullanıldıklarına dikkat edin.

English: Remember that type always comes first. Also, the methods that take type also take concurrency mode, so be wary of any question that only passes one option.

Türkçe: Type'ın her zaman önce geldiğini unutmayın. Type alan method'lar concurrency mode'u da alır; bu yüzden yalnızca tek option geçiren sorulara dikkat edin.

Controlling Data with Transactions

English: Until now, any changes we made to the database took effect right away. A commit is like saving a file. On the exam, changes commit automatically unless otherwise specified. However, you can change this behavior to control commits yourself. A transaction is when one or more statements are grouped with the final results committed or rolled back. Rollback is like closing a file without saving. All the changes from the start of the transaction are discarded. First, we look at writing code to commit and roll back. Then we look at how to control your rollback points.

Türkçe: Şimdiye kadar veritabanında yaptığımız değişiklikler hemen etkili oldu. `commit`, bir dosyayı kaydetmeye benzer; sınavda aksi belirtilmedikçe değişiklikler otomatik commit edilir. Bu davranışı değiştirip commit işlemlerini kendiniz yönetebilirsiniz. Transaction, bir veya daha çok statement'ın birlikte ele alınıp sonucun commit ya da rollback edildiği gruptur. `rollback`, dosyayı kaydetmeden kapatmaya benzer ve transaction başından itibaren yapılan değişiklikleri atar. Önce `commit()` ve `rollback()` code'unu, sonra rollback point'lerini yönetmeyi inceleyeceğiz.

Committing and Rolling Back

Dil çalışması: Bu başlıktaki kelimeler için [ünite sözlüğüne](#) bak.

English: Our zoo is renovating and has decided to give more space to the elephants. However, we only have so much space, so the zebra exhibit will need to be made smaller. Since we can't invent space out of thin air, we want to ensure that the total amount of space remains the same. If either adding space for the elephants or removing space for the zebras fails, we want our transaction to roll back. In the interest of simplicity, we assume that the database table is in a valid state before we run this code. Now, let's examine the code for this scenario:

Türkçe: Hayvanat bahçemiz yenileniyor ve fillere daha fazla alan ayırmaya karar verdi. Ancak toplam alanımız sınırlı olduğundan zebra exhibit'inin küçültülmesi gerekir. Yoktan alan yaratamayacağımız için toplam alanın aynı kalmasını isteriz. Fillere alan ekleme veya zebralardan alan çıkarma işlemlerinden biri başarısız olursa transaction'ı rollback etmek isteriz. Basitlik için bu code çalışmadan önce database table'nın geçerli durumda olduğunu varsayıyoruz. Şimdi bu senaryonun code'unu inceleyelim:

```
5: public static void main(String[] args) throws SQLException {
6:     try (Connection conn =
7:         DriverManager.getConnection("jdbc:hsqldb:file:zoo")) {
8:
9:         conn.setAutoCommit(false);
10:
11:         var elephantRowsUpdated = updateRow(conn, 5, "African Elephant");
```

```

12:         var zebraRowsUpdated = updateRow(conn, -5, "Zebra");
13:
14:         if (!elephantRowsUpdated || !zebraRowsUpdated)
15:             conn.rollback();
16:         else {
17:             String selectSql = ""
18:                 SELECT COUNT(*)
19:                 FROM exhibits
20:                 WHERE num_acres <= 0"";
21:             try (PreparedStatement ps = conn.prepareStatement(selectSql);
22:                 ResultSet rs = ps.executeQuery()) {
23:
24:                 rs.next();
25:                 int count = rs.getInt(1);
26:                 if (count == 0)
27:                     conn.commit();
28:                 else
29:                     conn.rollback();
30:             } } } }
31:
32: private static boolean updateRow(Connection conn,
33:     int numToAdd, String name)
34:
35:     throws SQLException {
36:
37:     String updateSql = ""
38:         UPDATE exhibits
39:         SET num_acres = num_acres + ?
40:         WHERE name = ?"";
41:
42:     try (PreparedStatement ps = conn.prepareStatement(updateSql)) {
43:         ps.setInt(1, numToAdd);
44:         ps.setString(2, name);
45:         return ps.executeUpdate() > 0;
46:     } }

```

English: The first interesting thing in this example is on line 9, where we turn off autocommit mode and declare that we will handle transactions ourselves. Most databases support disabling autocommit mode. If a database does not, it will throw a `SQLException` on line 9. We then attempt to update the number of acres allocated to each animal. If we are unsuccessful and no rows are updated, we roll back the transaction on line 15, causing the state of the database to remain unchanged.

Türkçe: Örnekteki ilk önemli nokta, autocommit mode'u kapatıp transaction'ları kendimiz yöneteceğimizi bildirdiğimiz 9. satırdır. Çoğu database autocommit'i devre dışı bırakmayı destekler; desteklemeyen bir database 9. satırda `SQLException` fırlatır. Ardından her animal'a ayrılan acre sayısını update etmeyi deneriz. İşlem başarısız olur ve hiç row güncellenmezse 15. satırda transaction'ı rollback eder, database state'ini değiştirmeden bırakırız.

English: Assuming at least one row is updated, we check exhibits and make sure none of the rows contain an invalid `num_acres` value. If this were a real application, we would have more logic to make sure the amount of space makes sense. On lines 26–30, we decide whether to commit the transaction to the database or roll back all updates made to the exhibits table.

Türkçe: En az bir row'un güncellendiğini varsayarsak `exhibits` table'ını kontrol edip hiçbir row'un geçersiz `num_acres` değeri içermediğinden emin oluruz. Bu gerçek bir uygulama olsaydı, alan miktarının mantıklı olduğundan emin olmak için daha fazla mantığa sahip olurduk. 26–30. satırlarda transaction'ı database'e commit etmeye veya `exhibits` table'ındaki tüm update'leri rollback etmeye karar veririz.

Autocommit Edge Cases

English: You need to know two edge cases for the exam. First, calling `setAutoCommit(true)` will automatically trigger a commit when you are not already in autocommit mode. After that, autocommit mode takes effect, and each statement is automatically committed.

Türkçe: Sınav için iki edge case'i bilmeniz gerekir. Önceden autocommit mode'da değilseniz `setAutoCommit(true)` çağrısı otomatik olarak commit tetikler. Bundan sonra autocommit mode etkinleşir ve her statement otomatik commit edilir.

English: The other edge case is what happens if you have autocommit set to false and close your connection without rolling back or committing your changes. The answer is that the behavior is undefined. It may commit or roll back, depending solely on the driver. Don't depend on this behavior; remember to commit or roll back at the end of a transaction!

Türkçe: Diğer edge case, autocommit `false` iken değişiklikleri `rollback()` veya `commit()` etmeden `Connection` kapatmaktır. Davranış tanımlı değildir; yalnızca driver'a bağlı olarak commit ya da rollback gerçekleşebilir. Buna güvenmeyin; transaction sonunda mutlaka explicit `commit()` veya `rollback()` çağırın.

JDBC contract: Buradaki "undefined" sonucu, driver-independent bir commit veya rollback garantisi olmadığı şeklinde oku. Active manual transaction kapatılmadan önce explicit `commit()` ya da `rollback()` çağır.

Bookmarking with Savepoints

English: So far, we have rolled back to the point where autocommit was turned off. You can use savepoints to have more control of the rollback point. Consider the following example:

Türkçe: Şimdiye kadar, autocommit kapatıldığı noktaya geri döndük. rollback noktasını daha fazla kontrol etmek için savepoints kullanabilirsiniz. Aşağıdaki örneği ele alalım:

```
20: conn.setAutoCommit(false);
21: Savepoint sp1 = conn.setSavepoint();
22: // database code
23: Savepoint sp2 = conn.setSavepoint("second savepoint");
24: // database code
25: conn.rollback(sp2);
26: // database code
27: conn.rollback(sp1);
```

English: Line 20 is important. You can only use savepoints when you are controlling the transaction. Lines 21 and 23 show how to create a Savepoint. The name is optional and typically included in the `toString()` if you print the savepoint reference.

Türkçe: 20. satır önemlidir: savepoint'leri yalnız transaction'ı kendiniz yönetirken kullanabilirsiniz. 21 ve 23. satırlar `Savepoint` oluşturmayı gösterir. Name optional'dır ve savepoint reference'ini yazdırırsanız genellikle `toString()` çıktısında görünür.

English: Line 25 shows the first rollback. That gets rid of any changes made since that savepoint was created: in this case, the code on line 24. Then line 27 shows the second rollback getting rid of the code on line 22.

Türkçe: 25. satırdaki ilk rollback, `sp2` oluşturulduktan sonra yapılan değişiklikleri—bu örnekte 24. satırdaki değişikliği—geri alır. 27. satırdaki ikinci rollback ise 22. satırdaki değişikliği geri alır.

English: Order matters. If we reversed lines 25 and 27, the code would throw an exception. Rolling back to `sp1` gets rid of any changes made after that, which includes the second savepoint! Similarly, calling `conn.rollback()` on line 25 would void both savepoints, and line 27 would again throw an exception.

Türkçe: Sıra önemlidir. 25 ve 27. satırları ters çevirirsek code exception fırlatır. `sp1`e rollback etmek, ikinci savepoint dahil ondan sonra yapılan değişiklikleri kaldırır. Benzer biçimde 25. satırda `conn.rollback()` çağırmak iki savepoint'i de invalidate eder; 27. satır yine exception fırlatır.

Reviewing Transaction APIs

English: There aren't many methods for working with transactions, but you need to know all of the ones in TABLE 15.9.

Türkçe: transactions ile çalışmak için çok fazla metod yoktur, ancak TABLE 15.9'daki tüm metodları bilmeniz gerekir.

English: TABLE 15.9 · Connection APIs for transactions.

Türkçe: TABLE 15.9 `Connection` transactions için API'ler.

Method	Description
<code>setAutoCommit(boolean b)</code>	Sets whether statements commit immediately
<code>commit()</code>	Saves data in the database
<code>rollback()</code>	Discards statements already made in the current transaction
<code>rollback(Savepoint sp)</code>	Goes back to the state at the <code>Savepoint</code>
<code>setSavepoint()</code>	Creates an unnamed bookmark
<code>setSavepoint(String name)</code>	Creates a named bookmark

Closing Database Resources

English: As you saw in Chapter 14, “I/O,” it is important to close resources when you are finished with them. This is true for JDBC as well. JDBC resources, such as a `Connection`, are expensive to create. Not closing them creates a resource leak that will eventually slow your program.

Türkçe: Chapter 14 “I/O”da gördüğümüz gibi resource'larla işiniz bittiğinde onları kapatmak önemlidir. Bu JDBC için de geçerlidir. `Connection` gibi JDBC resource'larının oluşturulması pahalıdır. Onları kapatmamak, sonunda programınızı yavaşlatacak bir kaynak sızıntısı yaratır.

English: Throughout the chapter, we've been using the try-with-resources syntax from Chapter 11. The resources need to be closed in a specific order. The `ResultSet` is closed first, followed by the `PreparedStatement` (or `CallableStatement`) and then the `Connection`.

Türkçe: Bölüm boyunca, Bölüm 11'den try-with-resources sözdizimi kullanıyoruz. Kaynakların belirli bir sırayla kapatılması gerekir. `ResultSet` önce kapatılır, ardından `PreparedStatement` (veya `CallableStatement`) ve daha sonra `Connection`.

English: While it is a good habit to close all three resources, it isn't strictly necessary. Closing a JDBC resource should close any resources that it created. In particular, the following are true:

Türkçe: Her üç kaynağı da kapatmak iyi bir alışkanlık olsa da, kesinlikle gerekli değildir. Bir JDBC kaynağının kapatılması, yarattığı herhangi bir kaynağı kapatmalıdır. Özellikle, aşağıdakiler doğrudur:

English: Closing a `Connection` also closes `PreparedStatement` (or `CallableStatement`) and `ResultSet`.

Türkçe: Bir `Connection` kapatıldığında onun oluşturduğu `PreparedStatement` (veya `CallableStatement`) ile `ResultSet` de kapanır.

English: Closing a `PreparedStatement` (or `CallableStatement`) also closes the `ResultSet`.

Türkçe: Bir `PreparedStatement` (veya `CallableStatement`) kapatıldığında onun `ResultSeti` de kapanır.

English: It is important to close resources in the right order. This avoids both resource leaks and exceptions.

Türkçe: Kaynakları doğru düzende kapatmak önemlidir. Bu, hem kaynak sızıntılarını hem de exception'ları önler.

Ownership ayrıntısı: `ResultSet` → `statement` → `Connection` reverse order'ı güvenli ve açık cleanup sırasıdır. Ancak her üçünü ayrı ayrı close etmek mutlak bir "must" değildir: `Connection.close()` ona bağlı `statement/result` resource'larını, `statement` close da current `ResultSeti` kapatır.

Writing a Resource Leak

English: In Chapter 11, you learned that it is possible to declare a type before a try-with-resources statement. Do you see why this method is bad?

Türkçe: Bölüm 11'de, bir try-with-resources statement'dan önce bir tür beyan etmenin mümkün olduğunu öğrendiniz. Bu metodun neden kötü olduğunu anlıyor musunuz?

```

40: public void bad() throws SQLException {
41:     var url = "jdbc:hsqldb:zoo";
42:     var sql = "SELECT not_a_column FROM names";
43:     var conn = DriverManager.getConnection(url);
44:     var ps = conn.prepareStatement(sql);
45:     var rs = ps.executeQuery();
46:
47:     try (conn; ps; rs) {
48:         while (rs.next())
49:             System.out.println(rs.getString(1));
50:     }
51: }

```

English: Suppose an exception is thrown on line 45. The try-with-resources block is never entered, so we don't benefit from automatic resource closing. That means this code has a resource leak if it fails. Do not write code like this.

Türkçe: 45. satırda bir exception fırlatıldığını varsayalım. Try-with-resources block'una hiç girilmediği için otomatik kaynak kapatma işleminden yararlanmıyoruz. Bu, bu kodun başarısız olması durumunda bir kaynak sızıntısı olduğu anlamına gelir. Böyle kod yazmayın.

English: There's another way to close a `ResultSet`. JDBC automatically closes a `ResultSet` when you run another SQL statement from the same `Statement`. This could be a `PreparedStatement` or a `CallableStatement`.

Türkçe: `ResultSet`i kapatmanın başka bir yolu daha vardır. JDBC, aynı `Statement` üzerinden başka bir SQL statement'ı çalıştırıldığında önceki `ResultSet`i otomatik kapatır. Bu `Statement`, `PreparedStatement` veya `CallableStatement` olabilir.

Dealing with Exceptions

English: In most of this chapter, we've lived in a perfect world. Sure, we mentioned that a checked `SQLException` might be thrown by any JDBC method—but we never caught it. We just declared it and let the caller deal with it. Now let's catch the exception.

Türkçe: Chapter'ın büyük bölümünde kusursuz bir dünyadaymış gibi ilerledik. Her JDBC method'unun checked `SQLException` fırlatabileceğini belirttik; fakat exception'ı hiç catch etmedik, yalnızca `throws` ile declare edip caller'a bıraktık. Şimdi exception'ı yakalayalım.

```

var sql = "SELECT not_a_column FROM names";
var url = "jdbc:hsqldb:zoo";
try (var conn = DriverManager.getConnection(url);
    var ps = conn.prepareStatement(sql);
    var rs = ps.executeQuery()) {

```

```

    while (rs.next())
        System.out.println(rs.getString(1));
} catch (SQLException e) {
    System.out.println(e.getMessage());
    System.out.println(e.getSQLState());
    System.out.println(e.getErrorCode());
}

```

English: The output looks like this:

Türkçe: Çıkış şu şekilde görünüyor:

```
Column 'NOT_A_COLUMN' is either not in any table...
42X04
30000
```

English: Each of these methods gives you a different piece of information. The `getMessage()` method returns a human-readable message about what went wrong. We've only included the beginning of it here. The `getSQLState()` method returns a code as to what went wrong. You can Google the name of your database and the SQL state to get more information about the error. In comparison, `getErrorCode()` is a database-specific code. On this database, it doesn't do anything.

Türkçe: Bu metodların her biri size farklı bir bilgi verir. `getMessage()` metodu, neyin yanlış gittiği hakkında insan tarafından okunabilir bir mesaj döndürür. Biz sadece başlangıcını buraya ekledik. `getSQLState()` metodu, neyin yanlış gittiğine dair bir kod döndürür. Hata hakkında daha fazla bilgi almak için veritabanınızın adını ve SQL durumunu Google'da bulabilirsiniz. Karşılaştırmada, `getErrorCode()` veritabanına özgü bir koddur. Bu veritabanında, hiçbir şey yapmaz.

Summary

English: There are four key SQL statements you should know for the exam, one for each of the CRUD operations: create (INSERT) a new row, read (SELECT) data, update (UPDATE) one or more rows, and delete (DELETE) one or more rows.

Türkçe: Her CRUD işlemi için bir temel SQL deyimini bilin: **INSERT** yeni satır ekler, **SELECT** veri okur, **UPDATE** bir veya daha fazla satırı günceller, **DELETE** bir veya daha fazla satırı siler.

English: For the exam, you should be familiar with five JDBC interfaces: `Driver`, `Connection`, `PreparedStatement`, `CallableStatement`, and `ResultSet`. The interfaces are part of the Java API. A database-specific JAR file provides the implementations.

Türkçe: Beş temel JDBC interface'ini tanıyın: **Driver**, **Connection**, **PreparedStatement**, **CallableStatement** ve **ResultSet**. Interface'ler Java API'sinin parçasıdır; somut gerçekleştirmeleri veritabanına özgü JAR dosyası sağlar.

English: To connect to a database, you need the JDBC URL. A JDBC URL has three parts separated by colons. The first part is `jdbc`. The second part is the name of the vendor/product. The third part varies by database, but it includes the location and/or name of the database. The location is either `localhost` or an IP address followed by an optional port.

Türkçe: Veritabanına bağlanmak için JDBC URL gerekir. URL, iki noktayla ayrılan üç ana bölümden oluşur: **jdbc**, sağlayıcı/ürün adı ve veritabanına özgü bölüm. Son bölüm konum ve/veya veritabanı adı içerir; sunucu adresi ve isteğe bağlı bağlantı noktası bulunabilir.

English: The DriverManager class provides a factory method called getConnection() to get a Connection implementation. You create a PreparedStatement or CallableStatement using prepareStatement() and prepareCall(), respectively. A PreparedStatement is used when the SQL is specified in your application, and a CallableStatement is used when the SQL is in the database. A PreparedStatement allows you to set the values of bind variables. A CallableStatement also allows you to set IN, OUT, and INOUT parameters.

Türkçe: DriverManager, bir Connection gerçekleştirimi edinmek için getConnection() üretici metodunu sunar. prepareStatement() ile PreparedStatement, prepareCall() ile CallableStatement oluşturulur. İlki uygulamada verilen SQL'i; ikincisi veritabanında saklanan yordamı çağırmak için kullanılır. PreparedStatement, bağlama değişkenlerine değer vermeyi; CallableStatement, ayrıca IN, OUT ve INOUT parametreleriyle çalışmayı sağlar.

English: When running a SELECT SQL statement, the executeQuery() method returns a ResultSet. When running a DELETE, INSERT, or UPDATE SQL statement, the executeUpdate() method returns the number of rows that were affected. There is also an execute() method that returns a boolean to indicate whether the statement was a query.

Türkçe: SELECT çalıştırıldığında executeQuery() bir ResultSet döndürür. DELETE, INSERT veya UPDATE için executeUpdate(), etkilenen satır sayısını döndürür. execute() ise ilk sonucun bir ResultSet olup olmadığını gösteren boolean değer döndürür.

English: You call rs.next() from an if statement or while loop to advance the cursor position. To get data from a column, call a method like getString(1) or getString("a"). Column indexes begin with 1, not 0. In addition to getting a String or primitive, you can call getObject() to get any type.

Türkçe: İmleci ilerletmek için if veya while içinde rs.next() çağrılır. Sütun değeri, getString(1) veya getString("a") gibi metotlarla okunur. Sütun indeksleri 0'dan değil 1'den başlar. String ve primitive değerleri okumanın yanı sıra getObject() ile nesne türünde değer alınabilir.

English: JDBC lets you choose whether to automatically commit your statements or manage transactions yourself. If you choose the latter, you can control when data is committed or rolled back. Additionally, you can set savepoints to roll back to specific points.

Türkçe: JDBC, deyimleri otomatik olarak kalıcılaştırmayı veya transaction'ları kendiniz yönetmeyi seçmenize olanak sağlar. İkinci durumda değişikliklerin ne zaman kalıcılaştırılacağını veya geri alınacağını siz belirlersiniz. Ayrıca belirli noktalara dönebilmek için kayıt noktaları oluşturabilirsiniz.

English: It is important to close JDBC resources when finished with them to avoid leaking resources. Closing a Connection automatically closes the Statement and ResultSet objects. Closing a Statement automatically closes the ResultSet object. Also, running another SQL statement closes the previous ResultSet object from that Statement.

Türkçe: Kaynak sızıntısını önlemek için işiniz bittiğinde JDBC kaynaklarını kapatın. Connection kapatıldığında ilişkili Statement ve ResultSet nesneleri; Statement kapatıldığında onun ResultSet nesnesi otomatik kapanır. Aynı Statement ile başka bir SQL deyimini çalıştırmak da önceki sonuç kümesini kapatır.

Exam Essentials

English: Name the core five JDBC interfaces that you need to know for the exam and where they are defined. The five key interfaces are `Driver`, `Connection`, `PreparedStatement`, `CallableStatement`, and `ResultSet`. The interfaces are part of the core Java APIs. The implementations are part of a database driver JAR file.

Türkçe: Beş temel JDBC interface'ini ve nerede tanımlandıklarını söyleyebilmelisiniz: `Driver`, `Connection`, `PreparedStatement`, `CallableStatement` ve `ResultSet`. Interface'ler Java API'sinde, gerçekleştirmeler veritabanı sürücüsünün JAR dosyasında bulunur.

English: Identify correct and incorrect JDBC URLs. A JDBC URL starts with `jdbc:`, followed by the vendor/product name. Next comes another colon and then a database-specific connection string. This database-specific string includes the location, such as `localhost` or an IP address with an optional port. It may also contain the name of the database.

Türkçe: Doğru ve yanlış JDBC URL biçimlerini ayırt edin. URL, `jdbc:` ile başlar; sağlayıcı/ürün adı ve bir iki noktadan sonra veritabanına özgü bağlantı metni gelir. Bu bölüm sunucu konumu, isteğe bağlı bağlantı noktası ve veritabanı adı içerebilir.

English: Describe how to get a `Connection` using `DriverManager`. After including the driver JAR in the classpath, call `DriverManager.getConnection(url)` or `DriverManager.getConnection(url, username, password)` to get a driver-specific `Connection` implementation class.

Türkçe: `DriverManager` ile bağlantı almayı açıklayın. Sürücü JAR dosyasını classpath üzerine ekledikten sonra `DriverManager.getConnection(url)` veya `DriverManager.getConnection(url, username, password)` çağırarak sürücüyü özgü bir `Connection` nesnesi alın.

English: Run queries using a `PreparedStatement`. When using a `PreparedStatement`, the SQL contains question marks (?) for the parameters or bind variables. This SQL is passed at the time the `PreparedStatement` is created, not when it is run. You must call a setter for each of these with the proper value before executing the query.

Türkçe: `PreparedStatement` ile sorgu çalıştırın. SQL içindeki parametreler soru işaretleriyle (?) gösterilir. SQL metni, deyim yürütülürken değil `PreparedStatement` oluşturulurken verilir. Çalıştırma öncesinde gerekli bütün bağlama değişkenlerine uygun setter metotlarıyla değer atanmış olmalıdır.

English: Run queries using a `CallableStatement`. When using a `CallableStatement`, the SQL looks like `{ call my_proc(?) }`. If you are returning a value, `{?= call my_proc(?)}` is also permitted. You must set any parameter values before executing the query. Additionally, you must call `registerOutParameter()` for any OUT or INOUT parameters.

Türkçe: `CallableStatement` ile saklı yordam çağırın. Temel sözdizimi `{ call my_proc(?) }` biçimindedir; bir fonksiyonun dönüş değeri alınacaksa `{?= call my_proc(?)}` biçimi kullanılır. Çalıştırmadan önce giriş değerlerini atayın. Her OUT veya INOUT parametresi için `registerOutParameter()` çağırın.

English: Loop through a `ResultSet`. Before trying to get data from a `ResultSet`, you call `rs.next()` inside an `if` statement or `while` loop. This ensures that the cursor is in a valid position. To get data from a column, call a method like `getString(1)` or `getString("a")`. Remember that column indexes begin with 1.

Türkçe: `ResultSet` üzerinde ilerleyin. Veri okumadan önce `if` veya `while` içinde `rs.next()` çağırın ve dönen `true` değeriyle geçerli satırda olduğunuzu doğrulayın. Sütunu `getString(1)` veya `getString("a")` gibi bir metotla okuyun. Sütun indekslerinin 1'den başladığını unutmayın.

English: Work with transactions. When `autocommit` is `false`, the `commit()` and `rollback()` methods control the transaction. There is an overloaded `rollback` method taking a `Savepoint` to roll back to a specific point.

Türkçe: Transaction'ları yönetebilmelisiniz. Otomatik kalıcılaştırma kapalıyken `commit()` ve `rollback()` sonucu belirler. `Savepoint` alan `rollback()` overload'u, yalnız belirtilen kayıt noktasına kadar geri dönmek için kullanılır.

English: Identify when a resource should be closed. If you're closing all three resources, the `ResultSet` must be closed first, followed by the `PreparedStatement` / `CallableStatement`, and the `Connection`.

Türkçe: Kaynakların ne zaman kapatılacağını belirleyin. Üç kaynak da açıkça kapatılacaksa sıra `ResultSet` → `PreparedStatement/CallableStatement` → `Connection` olmalıdır.

Review Questions

English: The answers to the chapter review questions can be found in the Appendix.

Türkçe: Bölüm sonu review sorularının cevapları Appendix'te bulunmaktadır.

Question 1 / Soru 1

English: 1. Which interfaces or classes are in a database-specific JAR file? (Choose all that apply.)

Türkçe: 1. Hangi interface veya class'lar database-specific bir JAR dosyasında bulunur? (Geçerli olanların tümünü seçin.)

English: A. `Driver`

Türkçe: A. `Driver`

English: B. `Driver's` implementation

Türkçe: B. `Driver` implementation'ı

English: C. `Manager`

Türkçe: C. `Manager`

English: D. DriverManager's implementation

Türkçe: D. `DriverManager` implementation'ı

English: E. PreparedStatement

Türkçe: E. `PreparedStatement`

English: F. PreparedStatement implementation

Türkçe: F. `PreparedStatement` implementation'ı

Question 2 / Soru 2

English: 2. Which of the following is a valid JDBC URL?

Türkçe: 2. Aşağıdakilerden hangisi geçerli bir JDBC URL'dir?

English: A. jdbc:sybase:localhost:1234/db

Türkçe: A. `jdbc:sybase:localhost:1234/db`

English: B. jdbc::sybase::localhost::/db

Türkçe: B. `jdbc::sybase::localhost::/db`

English: C. jdbc::sybase:localhost::1234/db

Türkçe: C. `jdbc::sybase:localhost::1234/db`

English: D. sybase:localhost:1234/db

Türkçe: D. `sybase:localhost:1234/db`

English: E. sybase::localhost::/db

Türkçe: E. `sybase::localhost::/db`

English: F. sybase::localhost::1234/db

Türkçe: F. `sybase::localhost::1234/db`

Question 3 / Soru 3

English: 3. Which of the options can fill in the blank to make the code compile and run without error? (Choose all that apply.)

Türkçe: 3. Code'un derlenmesini ve hata olmadan çalışmasını sağlamak için boşluğu hangi seçenekler doldurabilir? (Uygun olanların tümünü seçin.)

```
var sql = ""
    UPDATE habitat SET environment = null
    WHERE environment = ? "";
try (var ps = conn.prepareStatement(sql)) {
    _____
    ps.executeUpdate();
}
```

English: A. `ps.setString(0, "snow");`

Türkçe: A. `ps.setString(0, "snow");`

English: B. `ps.setString(1, "snow");`

Türkçe: B. `ps.setString(1, "snow");`

English: C. `ps.setString("environment", "snow");`

Türkçe: C. `ps.setString("environment", "snow");`

English: D. `ps.setString(1, "snow"); ps.setString(1, "snow");`

Türkçe: D. `ps.setString(1, "snow"); ps.setString(1, "snow");`

English: E. `ps.setString(1, "snow"); ps.setString(2, "snow");`

Türkçe: E. `ps.setString(1, "snow"); ps.setString(2, "snow");`

English: F. `ps.setString("environment", "snow"); ps.setString("environment", "snow");`

Türkçe: F. `ps.setString("environment", "snow"); ps.setString("environment", "snow");`

Question 4 / Soru 4

English: 4. Suppose that you have a table named `animal` with two rows. What is the result of the following code?

Türkçe: 4. `animal` adlı table'da iki row bulunduğunu varsayalım. Aşağıdaki code'un sonucu nedir?

```
6: var conn = new Connection(url, userName, password);
7: var ps = conn.prepareStatement(
8:     "SELECT count(*) FROM animal");
9: var rs = ps.executeQuery();
10: if (rs.next()) System.out.println(rs.getInt(1));
```

English: A. 0

Türkçe: A. 0

English: B. 2

Türkçe: B. 2

English: C. There is a compiler error on line 6.

Türkçe: C. 6. satırda compiler error vardır.

English: D. There is a compiler error on line 10.

Türkçe: D. 10. satırda compiler error vardır.

English: E. There is a compiler error on another line.

Türkçe: E. Başka bir satırda compiler error vardır.

English: F. A runtime exception is thrown.

Türkçe: F. Runtime'da exception fırlatılır.

Question 5 / Soru 5

English: 5. Which option can fill in the blanks to make the code compile?

Türkçe: 5. Code'un derlenmesi için boşlukları hangi seçenek doldurabilir?

```
boolean bool = ps._____.execute();
int num = ps._____.executeQuery();
ResultSet rs = ps._____.executeUpdate();
```

English: A. execute, executeQuery, executeUpdate

Türkçe: A. execute, executeQuery, executeUpdate

English: B. execute, executeUpdate, executeQuery

Türkçe: B. execute, executeUpdate, executeQuery

English: C. `executeQuery`, `execute`, `executeUpdate`

Türkçe: C. `executeQuery`, `execute`, `executeUpdate`

English: D. `executeQuery`, `executeUpdate`, `execute`

Türkçe: D. `executeQuery`, `executeUpdate`, `execute`

English: E. `executeUpdate`, `execute`, `executeQuery`

Türkçe: E. `executeUpdate`, `execute`, `executeQuery`

English: F. `executeUpdate`, `executeQuery`, `execute`

Türkçe: F. `executeUpdate`, `executeQuery`, `execute`

Question 6 / Soru 6

English: 6. Suppose there are two rows in the table before this code is run, and `executeUpdate()` runs without error. How many rows are in the table after the code completes?

Türkçe: 6. Bu code çalıştırılmadan önce table'da iki row bulunduğunu ve `executeUpdate()` çağrısının hata olmadan çalıştığını varsayalım. Code tamamlandığında table'da kaç row bulunur?

```
conn.setAutoCommit(true);

String sql = "INSERT INTO games VALUES(3, Jenga)";

try (PreparedStatement ps = conn.prepareStatement(sql,
    ResultSet.TYPE_FORWARD_ONLY, ResultSet.CONCUR_READ_ONLY)) {
    ps.executeUpdate();
}
conn.rollback();
```

English: A. Two

Türkçe: A. İki

English: B. Three

Türkçe: B. Üç

English: C. The code does not compile.

Türkçe: C. Kod derlenmez.

English: D. The code throws an exception.

Türkçe: D. Code exception fırlatır.

Question 7 / Soru 7

English: 7. Suppose that the table names has five rows and the following SQL statement updates all of them. What is the result of this code?

Türkçe: 7. `names` table'ında beş row bulunduğunu ve aşağıdaki SQL statement'ın bunların tümünü update ettiğini varsayalım. Bu code'un sonucu nedir?

```
public static void main(String[] args) throws SQLException {  
    var sql = "UPDATE names SET name = 'Animal'";  
    try (var conn = DriverManager.getConnection("jdbc:hsqldb:file:zoo");  
         var ps = conn.prepareStatement(sql)) {  
  
        var result = ps.executeUpdate();  
        System.out.println(result);  
    }  
}
```

English: A. 0

Türkçe: A. 0

English: B. 1

Türkçe: B. 1

English: C. 5

Türkçe: C. 5

English: D. The code does not compile.

Türkçe: D. Kod derlenmez.

English: E. A `SQLException` is thrown.

Türkçe: E. `SQLException` fırlatılır.

English: F. A different exception is thrown.

Türkçe: F. Farklı bir exception fırlatılır.

Question 8 / Soru 8

English: 8. Suppose learn() is a stored procedure that takes one IN parameter. What is wrong with the following code? (Choose all that apply.)

Türkçe: 8. `learn()` in bir **IN** parameter alan stored procedure olduğunu varsayalım. Aşağıdaki code'da ne yanlıştır? (Geçerli olanların tümünü seçin.)

```
18: var sql = "call learn()";
19: try (var cs = conn.prepareCall(sql)) {
20:     cs.setString(1, "java");
21:     try (var rs = cs.executeQuery()) {
22:         while (rs.next())
23:             System.out.println(rs.getString(3));
24:     }
25: }
```

English: A. Line 18 is missing braces.

Türkçe: A. 18. satırda braces (`{}`) eksiktir.

English: B. Line 18 is missing a ?.

Türkçe: B. 18. satırda bir `?` eksiktir.

English: C. Line 19 is not allowed to use var.

Türkçe: C. 19. satırda `var` kullanılmasına izin verilmez.

English: D. Line 20 does not compile.

Türkçe: D. 20. satır derlenmez.

English: E. Line 22 does not compile.

Türkçe: E. 22. satır derlenmez.

English: F. Something else is wrong with the code.

Türkçe: F. Code'da başka bir sorun vardır.

English: G. None of the above. This code is correct.

Türkçe: G. Yukarıdakilerin hiçbiri; bu code doğrudur.

Portable JDBC exam note: `{call learn(?)}` JDBC escape syntax'idir ve bu soruda beklenen biçimdir. Bazı driver'lar native callable syntax'i kabul edebilir; bu vendor-specific tolerans kaynak resmî **A**, **B** anahtarını değiştirmez.

Question 9 / Soru 9

English: 9. Suppose that the table enrichment has three rows with the animals bat, rat, and snake. How many lines does this code print?

Türkçe: 9. `enrichment` table'ında `bat`, `rat` ve `snake` hayvanlarını içeren üç row bulunduğunu varsayalım. Bu code kaç satır yazdırır?

```
var sql = "SELECT toy FROM enrichment WHERE animal = ?";
try (var ps = conn.prepareStatement(sql)) {
    try (var rs = ps.executeQuery()) {
        while (rs.next())
            System.out.println(rs.getString(1));
    }
}
```

English: A. 0

Türkçe: A. 0

English: B. 1

Türkçe: B. 1

English: C. 3

Türkçe: C. 3

English: D. The code does not compile.

Türkçe: D. Kod derlenmez.

English: E. A `SQLException` is thrown.

Türkçe: E. `SQLException` fırlatılır.

English: F. A different exception is thrown.

Türkçe: F. Farklı bir exception fırlatılır.

Question 10 / Soru 10

English: 10. Suppose that the table food has five rows, and this SQL statement updates all of them. What is the result of this code?

Türkçe: 10. `food` table'ında beş row bulunduğunu ve bu SQL statement'in bunların tümünü update ettiğini varsayalım. Bu code'un sonucu nedir?

```
public static void main(String[] args) {  
    var sql = "UPDATE food SET amount = amount + 1";  
    try (var conn = DriverManager.getConnection("jdbc:hsqldb:file:zoo");  
        var ps = conn.prepareStatement(sql)) {  
  
        var result = ps.executeUpdate();  
        System.out.println(result);  
    }  
}
```

English: A. 0

Türkçe: A. 0

English: B. 1

Türkçe: B. 1

English: C. 5

Türkçe: C. 5

English: D. The code does not compile.

Türkçe: D. Kod derlenmez.

English: E. A SQLException is thrown.

Türkçe: E. `SQLException` fırlatılır.

English: F. A different exception is thrown.

Türkçe: F. Farklı bir exception fırlatılır.

Question 11 / Soru 11

English: 11. Suppose we have a JDBC program that calls a stored procedure, which returns a set of results. Which is the correct order in which to close database resources for this call?

Türkçe: 11. Bir stored procedure çağıran ve sonuç kümesi döndüren bir JDBC programımız olduğunu varsayalım. Bu çağrıdaki database resource'larını kapatmanın doğru sırası hangisidir?

English: A. Connection, ResultSet, CallableStatement

Türkçe: A. `Connection`, `ResultSet`, `CallableStatement`

English: B. Connection, CallableStatement, ResultSet

Türkçe: B. `Connection`, `CallableStatement`, `ResultSet`

English: C. ResultSet, Connection, CallableStatement

Türkçe: C. `ResultSet`, `Connection`, `CallableStatement`

English: D. ResultSet, CallableStatement, Connection

Türkçe: D. `ResultSet`, `CallableStatement`, `Connection`

English: E. CallableStatement, Connection, ResultSet

Türkçe: E. `CallableStatement`, `Connection`, `ResultSet`

English: F. CallableStatement, ResultSet, Connection

Türkçe: F. `CallableStatement`, `ResultSet`, `Connection`

Question 12 / Soru 12

English: 12. Suppose that the table counts has five rows with the numbers 1 to 5. How many lines does this code print?

Türkçe: 12. `counts` table'ında 1–5 sayılarını içeren beş row bulunduğunu varsayalım. Bu code kaç satır yazdırır?

```
var sql = "SELECT num FROM counts WHERE num> ?";
try (var ps = conn.prepareStatement(sql,
    ResultSet.TYPE_SCROLL_SENSITIVE, ResultSet.CONCUR_UPDATABLE)) {
    ps.setInt(1, 3);

    try (var rs = ps.executeQuery()) {
        while (rs.next())
            System.out.println(rs.getObject(1));
    }

    ps.setInt(1, 100);

    try (var rs = ps.executeQuery()) {
        while (rs.next())
            System.out.println(rs.getObject(1));
    }
}
```

English: A. 0

Türkçe: A. 0

English: B. 1

Türkçe: B. 1

English: C. 2

Türkçe: C. 2

English: D. 4

Türkçe: D. 4

English: E. The code does not compile.

Türkçe: E. Kod derlenmez.

English: F. The code throws an exception.

Türkçe: F. Code exception fırlatır.

Question 13 / Soru 13

English: 13. Which of the following can fill in the blank correctly? (Choose all that apply.)

Türkçe: 13. Aşağıdakilerden hangileri boşluğu doğru biçimde doldurabilir? (Geçerli olanların tümünü seçin.)

```
var rs = ps.executeQuery();  
if (rs.next())  
    _____;
```

English: A. String s = rs.getString(0)

Türkçe: A. `String s = rs.getString(0)`

English: B. String s = rs.getString(1)

Türkçe: B. `String s = rs.getString(1)`

English: C. String s = rs.getObject(0)

Türkçe: C. `String s = rs.getObject(0)`

English: D. String s = rs.getObject(1)

Türkçe: D. `String s = rs.getObject(1)`

English: E. Object s = rs.getObject(0)

Türkçe: E. `Object s = rs.getObject(0)`

English: F. Object s = rs.getObject(1)

Türkçe: F. `Object s = rs.getObject(1)`

Question 14 / Soru 14

English: 14. Suppose learn() is a stored procedure that takes one IN parameter and one OUT parameter. What is wrong with the following code? (Choose all that apply.)

Türkçe: 14. `learn()` in bir **IN** ve bir **OUT** parameter alan stored procedure olduğunu varsayalım. Aşağıdaki code'da ne yanlıştır? (Geçerli olanların tümünü seçin.)

```
18: var sql = "{?= call learn(?)}";
19: try (var cs = conn.prepareStatement(sql)) {
20:     cs.setInt(1, 8);
21:     cs.execute();
22:     System.out.println(cs.getInt(1));
23: }
```

English: A. Line 18 does not call the stored procedure properly.

Türkçe: A. 18. satır stored procedure'ü doğru biçimde çağırılmaz.

English: B. The parameter value is not set for input.

Türkçe: B. Input için parameter value set edilmemiştir.

English: C. The parameter is not registered for output.

Türkçe: C. Parameter output için register edilmemiştir.

English: D. The code does not compile.

Türkçe: D. Kod derlenmez.

English: E. Something else is wrong with the code.

Türkçe: E. Code'da başka bir sorun vardır.

English: F. None of the above. This code is correct.

Türkçe: F. Yukarıdakilerin hiçbiri; bu code doğrudur.

Question 15 / Soru 15

English: 15. Which can fill in the blank and have the code run without error? (Choose all that apply.)

Türkçe: 15. Hangileri boşluğu doldurup code'un hata olmadan çalışmasını sağlayabilir? (Geçerli olanların tümünü seçin.)

```
17: conn.setAutoCommit(false);
18:
19: var larry = conn.setSavepoint();
20: var curly = conn.setSavepoint();
21: var moe = conn.setSavepoint();
22: var shemp = conn.setSavepoint();
23:
24: _____;
25:
26: conn.rollback(curly);
```

English: A. `conn.rollback(larry)`

Türkçe: A. `conn.rollback(larry)`

English: B. `conn.rollback(curly)`

Türkçe: B. `conn.rollback(curly)`

English: C. `conn.rollback(moe)`

Türkçe: C. `conn.rollback(moe)`

English: D. `conn.rollback(shemp)`

Türkçe: D. `conn.rollback(shemp)`

English: E. `conn.rollback()`

Türkçe: E. `conn.rollback()`

English: F. The code does not compile.

Türkçe: F. Kod derlenmez.

Question 16 / Soru 16

English: 16. Which of the following can fill in the blank? (Choose all that apply.)

Türkçe: 16. Aşağıdakilerden hangileri boşluğu doldurabilir? (Geçerli olanların tümünü seçin.)

```
var sql = "_____";
try (var ps = conn.prepareStatement(sql)) {
    ps.setObject(3, "red");
    ps.setInt(2, 8);
    ps.setString(1, "ball");
    ps.executeUpdate();
}
```

English: A. { call insert_toys(?,?) }

Türkçe: A. { call insert_toys(?,?) }

English: B. { call insert_toys(?,?,?) }

Türkçe: B. { call insert_toys(?,?,?) }

English: C. { call insert_toys(?,?,?,?) }

Türkçe: C. { call insert_toys(?,?,?,?) }

English: D. INSERT INTO toys VALUES (?,?)

Türkçe: D. INSERT INTO toys VALUES (?,?)

English: E. INSERT INTO toys VALUES (?,?,?)

Türkçe: E. INSERT INTO toys VALUES (?,?,?)

English: F. INSERT INTO toys VALUES (?,?,?,?)

Türkçe: F. INSERT INTO toys VALUES (?,?,?,?)

Question 17 / Soru 17

English: 17. Suppose that the table counts has five rows with the numbers 1 to 5. How many lines does this code print?

Türkçe: 17. counts table'ında 1–5 sayılarını içeren beş row bulunduğunu varsayalım. Bu code kaç satır yazdırır?

```
var sql = "SELECT num FROM counts WHERE num> ?";
try (var ps = conn.prepareStatement(sql)) {
    ps.setInt(1, 3);

    try (var rs = ps.executeQuery()) {
        while (rs.next())
            System.out.println(rs.getObject(1));
    }
    try (var rs = ps.executeQuery()) {
        while (rs.next())
            System.out.println(rs.getObject(1));
    }
}
```

English: A. 0

Türkçe: A. 0

English: B. 1

Türkçe: B. 1

English: C. 2

Türkçe: C. 2

English: D. 4

Türkçe: D. 4

English: E. The code does not compile.

Türkçe: E. Kod derlenmez.

English: F. The code throws an exception.

Türkçe: F. Code bir exception fırlatır.

Question 18 / Soru 18

English: 18. There are currently 100 rows in the table species before inserting a new row. What is the output of the following code?

Türkçe: 18. Yeni bir row eklenmeden önce `species` table'ında 100 row bulunmaktadır. Aşağıdaki code'un çıktısı nedir?

```
String insert = "INSERT INTO species VALUES (3, 'Ant', .05)";
String select = "SELECT count(*) FROM species";
try (var ps = conn.prepareStatement(insert)) {
    ps.executeUpdate();
}
try (var ps = conn.prepareStatement(select)) {
    var rs = ps.executeQuery();
    System.out.println(rs.getInt(1));
}
```

English: A. 100

Türkçe: A. 100

English: B. 101

Türkçe: B. 101

English: C. The code does not compile.

Türkçe: C. Kod derlenmez.

English: D. A `SQLException` is thrown.

Türkçe: D. `SQLException` fırlatılır.

English: E. A different exception is thrown.

Türkçe: E. Farklı bir exception fırlatılır.

Question 19 / Soru 19

English: 19. Which of the options can fill in the blank to make the code compile and run without error? (Choose all that apply.)

Türkçe: 19. Aşağıdaki seçeneklerden hangileri boşluğu doldurduğunda code derlenir ve hata vermeden çalışır? (Uygun olanların tümünü seçin.)

```
var sql = "UPDATE habitat WHERE environment = ?";
try (var ps = conn.prepareStatement(sql)) {
    _____
    ps.executeUpdate();
}
```

English: A. `ps.setString(0, "snow");`

Türkçe: A. `ps.setString(0, "snow");`

English: B. `ps.setString(1, "snow");`

Türkçe: B. `ps.setString(1, "snow");`

English: C. `ps.setString("environment", "snow");`

Türkçe: C. `ps.setString("environment", "snow");`

English: D. The code does not compile.

Türkçe: D. Kod derlenmez.

English: E. The code throws an exception at runtime.

Türkçe: E. Code çalışma zamanında bir exception fırlatır.

Question 20 / Soru 20

English: 20. Which is the first line containing a compiler error?

Türkçe: 20. Derleyici hatası içeren ilk satır hangisidir?

```
25: String url = "jdbc:hsqldb:file:zoo";
26: try (var conn = DriverManager.getConnection(url);
27:     var ps = conn.prepareStatement();
28:     var rs = ps.executeQuery("SELECT * FROM swings")) {
29:     while (rs.next()) {
30:         System.out.println(rs.getInteger(1));
31:     }
32: }
```

English: A. Line 26

Türkçe: A. 26. satır

English: B. Line 27

Türkçe: B. 27. satır

English: C. Line 28

Türkçe: C. 28. satır

English: D. Line 29

Türkçe: D. 29. satır

English: E. Line 30

Türkçe: E. 30. satır

English: F. None of the above

Türkçe: F. Yukarıdakilerin hiçbiri

Question 21 / Soru 21

English: 21. Suppose conn is a valid connection object and the exhibits table is empty. Which are true? (Choose two.)

Türkçe: 21. `conn` değişkeninin geçerli bir `Connection` object'i olduğunu ve `exhibits` table'ının boş olduğunu varsayalım. Hangileri doğrudur? (İkisini seçin.)

```
try (conn) {  
    conn.setAutoCommit(false);  
    String sql = "INSERT INTO exhibits VALUES(3, 'Test', 2)";  
    try (PreparedStatement ps = conn.prepareStatement(sql)) {  
        ps.executeUpdate();  
    }  
    conn.setAutoCommit(true); // line W  
}
```

English: A. As written, the table will remain empty after this code.

Türkçe: A. Code mevcut haliyle çalıştırıldığında table boş kalır.

English: B. As written, the table will contain one row after this code.

Türkçe: B. Code mevcut haliyle çalıştırıldığında table bir row içerir.

English: C. As written, the code will throw an exception.

Türkçe: C. Code mevcut haliyle çalıştırıldığında bir exception fırlatır.

English: D. When line W is commented out, the table will remain empty after this code.

Türkçe: D. W satırı comment out edildiğinde code'dan sonra table boş kalır.

English: E. When line W is commented out, the table will contain one row after this code.

Türkçe: E. W satırı comment out edildiğinde code'dan sonra table bir row içerir.

English: F. When line W is commented out, the code will throw an exception.

Türkçe: F. W satırı comment out edildiğinde code bir exception fırlatır.

Appendix · Official Review Question Answers / Resmî Cevaplar

Aşağıdaki cevaplar kaynak Appendix bölümündeki sıra ve gerekçeleri korur. Türkçe bloklar doğal teknik çeviridir.

Official Answer 1

English: 1. B, F. The Driver and PreparedStatement interfaces are part of the JDK, making options A and E incorrect. Option C is incorrect because we made it up. The concrete DriverManager class is also part of the JDK, making option D incorrect. Options B and F are correct since the implementation of these interfaces is part of the database-specific driver JAR file.

Türkçe: 1. B, F. `Driver` ve `PreparedStatement` interface'leri JDK'nın parçasıdır; dolayısıyla A ve E yanlıştır. C'deki `Manager` uydurma bir type'tır. Concrete `DriverManager` class'ı da JDK'da yer aldığı için D yanlıştır. Bu interface'lerin implementation'ları veritabanına özgü driver JAR'ında bulunduğundan B ve F doğrudur.

Official Answer 2

English: 2. A. A JDBC URL has three main parts separated by single colons, making options B, C, E, and F incorrect. The first part is always jdbc, making option D incorrect. Therefore, the correct answer is option A. Notice that you can get this right even if you've never heard of the Sybase database before.

Türkçe: 2. A. JDBC URL, tek colon'larla ayrılan üç ana parçadan oluşur; bu nedenle B, C, E ve F yanlıştır. İlk parça her zaman `jdbc` olduğundan D de yanlıştır. Doğru cevap A'dır. Sybase veritabanını daha önce hiç duymamış olsanız bile bu soruyu doğru yanıtlayabilirsiniz.

Official Answer 3

English: 3. B, D. When setting parameters on a PreparedStatement, there are only options that take an index, making options C and F incorrect. The indexing starts with 1, making option A incorrect. This query has only one parameter, so option E is also incorrect. Option B is correct because it simply sets the parameter. Option D is also correct because it sets the parameter and then immediately overwrites it with the same value.

Türkçe: 3. B, D. `PreparedStatement` parameter'ları yalnız index alan overload'larla ayarlanabildiğinden C ve F yanlıştır. Index 1'den başladığı için A yanlıştır. Query yalnızca bir parameter içerdiğinden E de yanlıştır. B parameter'ı bir kez ayarlar; D ise ayarlayıp hemen aynı değerle üzerine yazar. Bu nedenle B ve D doğrudur.

Official Answer 4

English: 4. C. A Connection is created using a static method on DriverManager. It does not use a constructor. Therefore, option C is correct. If the Connection was created properly, the answer would be option B.

Türkçe: 4. C. `Connection`, `DriverManager` üzerindeki static method ile oluşturulur; constructor kullanılmaz. Bu nedenle C doğrudur. `Connection` doğru oluşturulsaydı cevap B olurdu.

Official Answer 5

English: 5. B. The first line has a return type of boolean, making it an execute() call. The second line returns the number of modified rows, making it an executeUpdate() call. The third line returns the results of a query, making it an executeQuery() call. Therefore, option B is the answer.

Türkçe: 5. B. İlk satırın return type'ı `boolean` olduğundan çağrı `execute()` dur. İkinci satır değiştirilen row sayısını döndürdüğü için `executeUpdate()`; üçüncü satır query sonuçlarını döndürdüğü için `executeQuery()` çağrısıdır. Doğru cevap B'dir.

Official Answer 6

English: 6. B. The first line enables autocommit mode. This is the default and means to commit immediately after each update. When the rollback() runs, there are no uncommitted statements, so there is nothing to roll back. This gives us the initial two rows in addition to the inserted one making option B correct. If setAutoCommit(false) were called, option A would be the answer. The ResultSet types are just there to mislead you. Any types are valid for executeUpdate() since no ResultSet is involved.

Türkçe: 6. B. İlk satır default olan autocommit mode'u etkinleştirir; her update hemen commit edilir. `rollback()` çalıştığında uncommitted statement kalmadığından geri alınacak bir şey yoktur. Başlangıçtaki iki row'a eklenen row da kalır; böylece B doğrudur. `setAutoCommit(false)` çağrılseydi cevap A olurdu. `ResultSet` type'ları yalnızca yanıtıma amaçlıdır; `executeUpdate()` bir `ResultSet` üretmediği için burada tüm bu type'lar geçerlidir.

Official Answer 7

English: 7. C. This code works as expected. It updates each of the five rows in the table and returns the number of rows updated. Therefore, option C is correct.

Türkçe: 7. C. Bu kod beklendiği gibi çalışır. Tablodaki beş satırın her birini günceller ve güncellenen satır sayısını döndürür. Bu nedenle, C seçeneği doğrudur.

Official Answer 8

English: 8. A, B. Option A is one of the answers because you are supposed to use braces ({}) for all SQL in a CallableStatement. Option B is the other answer because each parameter should be passed with a question mark (?). The rest of the code is correct. Note that your database might not behave the way that's described here, but you still need to know this syntax for the exam.

Türkçe: 8. A, B. `CallableStatement` içindeki SQL, braces ({}) kullanması gerektiği için A doğrudur. Her parameter question mark (?) ile gösterilmesi gerektiği için B de doğrudur. Code'un geri kalanı doğrudur. Veritabanınız burada anlatılardan farklı davranabilse de sınav için bu syntax'ı bilmeniz gerekir.

Kaynak anahtarı / portable syntax: Resmî cevap **A, B** olarak korunur. `{call ...}` biçimi JDBC escape syntax'idir ve portable sınav beklentisidir. Bir driver'ın kendi native procedure syntax'ini ayrıca kabul etmesi vendor-specific davranıştır.

Official Answer 9

English: 9. E. This code declares a bind variable with ? but never assigns a value to it. The compiler does not enforce bind variables have values, so the code compiles, but produces a `SQLException` at runtime, making option E correct.

Türkçe: 9. E. Code, ? ile bir bind variable tanımlar fakat ona hiç değer atamaz. Compiler bind variable'ların değer almasını zorunlu kılmadığından code derlenir; runtime'da `SQLException` fırlatır. Bu nedenle E doğrudur.

Official Answer 10

English: 10. D. JDBC code throws a `SQLException`, which is a checked exception. The code does not handle or declare this exception, and therefore it doesn't compile. Since the code doesn't compile, option D is correct. If the exception were handled or declared, the answer would be option C.

Türkçe: 10. D. JDBC code'u checked exception olan `SQLException` fırlatabilir. Code bu exception'ı handle veya declare etmediğinden derlenmez; dolayısıyla D doğrudur. Exception handle ya da declare edilseydi cevap C olurdu.

Official Answer 11

English: 11. D. JDBC resources should be closed in the reverse order from that in which they were opened. The order for opening is `Connection`, `CallableStatement`, and `ResultSet`.

Türkçe: 11. D. JDBC resource'ları açılma sırasının tersine kapatılmalıdır. Açılış sırası `Connection`, `CallableStatement`, `ResultSet` tir.

English: The order for closing is `ResultSet`, `CallableStatement`, and `Connection`, which is option D.

Türkçe: Kapatma sırası `ResultSet`, `CallableStatement`, `Connection` dir; yani cevap D'dir.

Official Answer 12

English: 12. C. This code calls the `PreparedStatement` twice. The first time, it gets the numbers greater than 3. Since there are two such numbers, it prints two lines. The second time, it gets the numbers greater than 100. There are no such numbers, so the `ResultSet` is empty. Two lines are printed in total, making option C correct. The `ResultSet` options are just there to trick you since only the default settings are used by the rest of the code.

Türkçe: 12. C. Code aynı `PreparedStatement` i iki kez çalıştırır. İlk çalıştırma 3'ten büyük iki sayıyı aldığı için iki satır yazdırır. İkinci çalıştırma 100'den büyük sayı bulamaz; `ResultSet` boş olur. Toplam iki satır yazdırıldığından C doğrudur. Code'un geri kalanı yalnız default setting'leri kullandığı için `ResultSet` option'ları yanıltma amaçlıdır.

Official Answer 13

English: 13. B, F. In a `ResultSet`, columns are indexed starting with 1, not 0. Therefore, options A, C, and E are incorrect. There are methods to get the column as a `String` or `Object`. However, option D is incorrect because an `Object` cannot be assigned to a `String` without a cast.

Türkçe: 13. B, F. `ResultSet` column index'leri 0'dan değil 1'den başlar; bu nedenle A, C ve E yanlıştır. Column'u `String` veya `Object` olarak alan method'lar vardır. Ancak bir `Object`, cast yapılmadan `String` atanamayacağı için D yanlıştır.

Official Answer 14

English: 14. C. Since an OUT parameter is used, the code should call `registerOutParameter()`. Since this is missing, option C is correct.

Türkçe: 14. C. OUT parameter kullanıldığı için code `registerOutParameter()` çağırılmalıdır. Çağrı eksik olduğundan C doğrudur.

Java 17/JDBC escape-syntax düzeltmesi: Kaynak resmî anahtarı **C** olarak korunmuştur. Leading `{?= call ...}` placeholder'ı generic procedure OUT parameter'ı değil, function return value'sudur. Sorunun "bir IN ve bir OUT parameter alan stored procedure" öncülü portable biçimde `{call learn(?,?)}` gerektirir; ilgili input set edilmeli ve output `registerOutParameter()` ile register edilmelidir. Bu nedenle exact JDBC grammar açısından line 18 ve parameter-position modelinde de sorun vardır.

Official Answer 15

English: 15. C, D. Rolling back to a point invalidates any savepoints created after it. Options A and E are incorrect because they roll back to lines 19 and 17, respectively. Option B is incorrect because you cannot roll back to the same savepoint twice. Options C and D are the answers because those savepoints were created after curly.

Türkçe: 15. C, D. Bir noktaya rollback etmek, ondan sonra oluşturulan savepoint'leri invalidate eder. A ve E sırasıyla 19 ve 17. satırlardaki noktalara rollback ettiği için yanlıştır. Kaynağa göre B, aynı savepoint'e iki kez rollback edilemeyeceği için yanlıştır. C ve D'deki savepoint'ler curlyden sonra oluşturulduğu için kaynak cevabı C ve D'dir.

Java 17/JDBC contract düzeltmesi: Kaynak resmî anahtarı **C, D** olarak korunmuştur.

`rollback(curly)`, curly savepoint'ini otomatik olarak release etmez; JDBC `Connection.rollback(Savepoint)` contract'ında aynı valid savepoint'e ikinci kez rollback yapmayı yasaklayan bir kural yoktur. İlk placeholder'a B yazıldığında line 26'daki ikinci `rollback(curly)` de portable olarak geçerlidir. Bu nedenle exact Java/JDBC contract'a göre savunulabilir seçenek kümesi **B, C, D**'dir. A ve E, curly dahil sonraki savepoint'leri geçersiz kıldığı için başarısız olabilir.

Official Answer 16

English: 16. E. First, notice that this code uses a `PreparedStatement`. Options A, B, and C are incorrect because they are for a `CallableStatement`. Next, remember that the number of parameters must be an exact match, making option E correct. Remember that you will not be tested on SQL syntax. When you see a question that appears to be about SQL, think about what it might be trying to test you on.

Türkçe: 16. E. Önce code'un `PreparedStatement` kullandığına dikkat edin. A, B ve C `CallableStatement` syntax'ı olduğundan yanlıştır. Parameter sayısı tam olarak eşleşmelidir; bu nedenle E doğrudur. SQL syntax'ının sınanmayacağını unutmayın. SQL sorusu gibi görünen bir soruda aslında hangi Java/JDBC kuralının ölçüldüğünü düşünün.

Official Answer 17

English: 17. D. This code calls the `PreparedStatement` twice. The first time, it gets the numbers greater than 3. Since there are two such numbers, it prints two lines. Since the parameter is not set between the first and second calls, the second attempt also prints two rows. Four lines are printed in total, making option D correct.

Türkçe: 17. D. Code aynı `PreparedStatement` i iki kez çalıştırır. İlkinde 3'ten büyük iki sayı alınır ve iki satır yazdırılır. İlk ve ikinci çağrı arasında parameter değiştirilmediği için ikinci çalıştırma da iki satır yazdırır. Toplam dört satır yazdırıldığından D doğrudur.

Official Answer 18

English: 18. D. Before accessing data from a `ResultSet`, the cursor needs to be positioned. The call to `rs.next()` is missing from this code causing a `SQLException` and option D to be correct.

Türkçe: 18. D. `ResultSet` data'sına erişmeden önce cursor geçerli bir row'a konumlandırılmalıdır. Code'da `rs.next()` çağrısı eksik olduğu için `SQLException` fırlatılır; D doğrudur.

Official Answer 19

English: 19. E. This code should call `prepareStatement()` instead of `prepareCall()` since it is not executing a stored procedure. Since we are using `var`, it does compile. Java will happily create a `CallableStatement` for you. Since this compile safety is lost, the code will not cause issues until runtime. At that point, Java will complain that you are trying to execute SQL as if it were a stored procedure, making option E correct.

Türkçe: 19. E. Stored procedure çalıştırılmadığı için code `prepareCall()` yerine `prepareStatement()` çağırılmalıdır. `var` kullanıldığından code derlenir ve Java bir `CallableStatement` oluşturur. Compile-time type safety kaybolduğu için sorun runtime'a kadar görünmez. O aşamada SQL, stored procedure gibi çalıştırılmaya çalışıldığından exception oluşur; kaynak cevabı E'dir.

Driver-dependent failure point: Resmî cevap E korunur. `prepareCall()` normal `UPDATE` SQL'i için portable API seçimi değildir. Driver metni `prepareCall()` aşamasında da, execution aşamasında da reddedebilir; exact failure point ve message Java language guarantee değildir.

Official Answer 20

English: 20. B. The `prepareStatement()` method requires SQL to be passed in. Since this parameter is omitted, line 27 does not compile, and option B is correct.

Türkçe: 20. B. `prepareStatement()` method'una SQL geçirilmesi zorunludur. Bu argument verilmediğinden 27. satır derlenmez; B doğrudur.

Official Answer 21

English: 21. B, D. The code starts with autocommit off. As written, we turn autocommit mode back on and immediately commit the transaction. This is option B. When line W is commented out, the update gets lost, making option D the other answer.

Türkçe: 21. B, D. Code autocommit kapalı başlar. Mevcut haliyle autocommit tekrar açılır ve transaction hemen commit edilir; bu B'dir. W satırı comment out edildiğinde kaynağa göre update kaybolur ve diğer cevap D olur.

Java 17/JDBC contract düzeltmesi: Kaynak resmî anahtarı **B, D** olarak korunmuştur. B portably doğrudur; `setAutoCommit(true)` current transaction'ı commit eder. Line W çıkarıldığında ise try-with-resources `Connection.close()` çağrısını active manual transaction üzerinde yapar. JDBC contract bu durumda commit/rollback sonucunu implementation-defined bırakır. Dolayısıyla D, driver-independent garanti değildir; production code close öncesinde explicit `commit()` veya `rollback()` çağırılmalıdır.

Kapsam doğrulaması

- Ana bölüm kaynak sayfaları: 863–908
- Ek cevap kaynağı sayfaları: 959–961
- Resmî cevap hedefi: 1–21
- Kod blokları özgün dilinde tutulmuştur.
- Çeviri ayrıntıları ünite vocabulary ve grammar kaynaklarıyla desteklenir.